

INFRAGUARD AI: AN LLM-DRIVEN OBSERVABILITY AND INCIDENT-TRIAGE
AGENT FOR SELF-HOSTED CLOUD-NATIVE INFRASTRUCTURE

BY

ADEBODUN SAMUEL TOMOLA

MATRICULATION NUMBER: 2025/A/MIT/0616

DEPARTMENT OF INFORMATION TECHNOLOGY

SCHOOL OF COMPUTING

MIVA OPEN UNIVERSITY, ABUJA

NIGERIA

A PROFESSIONAL MASTER'S PROJECT SUBMITTED TO THE DEPARTMENT OF
INFORMATION TECHNOLOGY, SCHOOL OF COMPUTING, MIVA OPEN
UNIVERSITY, ABUJA, NIGERIA.

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE
PROFESSIONAL MASTER OF INFORMATION TECHNOLOGY DEGREE

MAY 2026

CERTIFICATION

I certify that this project is my original work, except where the work of others is acknowledged through citations and references. The project has not been submitted to this University or another institution for the award of a degree.

Student: Adebodun Samuel Tomola

Matriculation Number: 2025/A/MIT/0616

Signature: _____

Date: _____

APPROVAL

This project has been approved for the Department of Information Technology, School of Computing, Miva Open University, Abuja, Nigeria.

Project Supervisor: Dr. Emmanuel Mkpojiogu

Signature: _____ Date: _____

Head of Department: Dr. Abdullahi Mohammed

Signature: _____ Date: _____

Dean, School of Computing: Prof. Victoria Opara

Signature: _____ Date: _____

External Examiner: Prof. Kingsley Obani

Signature: _____ Date: _____

DEDICATION

This project is dedicated to my family and to the engineers who maintain critical digital services with limited operational resources.

ACKNOWLEDGEMENT

I thank Almighty God for the strength and opportunity to complete this programme and project.

I am grateful to my project supervisor and the staff of the Department of Information Technology for their guidance and criticism throughout the work. Their feedback helped narrow the project to a testable implementation.

I also thank colleagues and practitioners who discussed the operational problems that motivated the system, particularly alert fatigue, slow incident triage, and the cost of maintaining several monitoring tools.

I acknowledge the maintainers of the open-source software used in the implementation, including FastAPI, LangChain, LangGraph, ChromaDB, Prometheus, Grafana Loki, and CrowdSec.

Finally, I thank my family and friends for their patience, encouragement, and support during the programme.

ABSTRACT

Cloud-native systems produce logs, metrics, and service events faster than small operations teams can interpret them. Monitoring tools can detect abnormal signals, but an engineer must still correlate the available evidence, identify a likely cause, and decide what to do. This project develops InfraGuard AI, a self-hosted observability and incident-triage system intended to support that first review.

InfraGuard AI is a self-hosted observability assistant designed to turn operational telemetry into concise, structured incident guidance. The system collects logs from Loki, metrics from Prometheus, HTTP probe results, Docker events and CrowdSec signals, then applies a bounded collect–analyse–decide–notify workflow. Its retrieval component indexes local Markdown runbooks with an ONNX sentence-embedding model and ChromaDB. Model analysis can be supplied through the Gemini Developer API, Anthropic, an OpenAI-compatible endpoint, or a locally hosted Ollama model. This design keeps the operational data path explicit while allowing organisations to choose between hosted and self-managed inference.

Verification on 16 August 2026 produced 82 passing automated tests in 27.01 seconds, with 66% overall line coverage across 1,476 statements. The acknowledgement-memory module reached 100% coverage and the orchestration module reached 95%. A limited deployed pilot also confirmed that twenty local runbooks could be indexed and used to answer three illustrative questions on disk capacity, memory pressure and Loki ingestion. These results verify the main software paths and the live retrieval integration, but they do not constitute a controlled measurement of diagnostic accuracy, response cost, latency or practitioner usability.

Keywords: AIOps, observability, large language models, retrieval-augmented generation, site reliability engineering, incident triage, operator acknowledgement, threat detection, LangGraph.

TABLE OF CONTENTS

CHAPTER ONE: INTRODUCTION.....	1
1.1 Background to the Project.....	1
1.2 Statement of the Problem.....	2
1.3 Aim and Objectives of the Project.....	2
1.3.1 Aim.....	2
1.3.2 Specific Objectives.....	2
1.4 Scope of the Project.....	3
1.4.1 Within Scope.....	3
1.4.2 Out of Scope (Identified as Future Work).....	3
1.5 Significance of the Project.....	4
1.5.1 Significance to Industry.....	4
1.5.2 Significance to Organisations.....	4
1.5.3 Significance to Policy and Society.....	4
1.5.4 Significance to Professional Practice.....	4
1.5.5 Significance to the Master of Information Technology Programme.....	5
CHAPTER TWO: LITERATURE REVIEW AND TECHNOLOGY CONTEXT.....	6
2.1 Conceptual Review.....	6
2.1.1 Cloud-Native Observability and the Three Pillars.....	6
2.1.2 Site Reliability Engineering and Incident Management.....	6
2.1.3 AIOps: Artificial Intelligence for IT Operations.....	6
2.1.4 Large Language Models and Agentic AI.....	6
2.1.5 Retrieval-Augmented Generation.....	7
2.1.6 Statelessness, Alert Deduplication, and Agent Memory.....	7
2.2 Review of Existing Systems and Solutions.....	7
2.2.1 Comparative Table of Thirteen Recent and Related Studies.....	7
2.2.2 Synthesis of the Reviewed Studies.....	12
2.2.3 Commercial and Open-Source State of the Art.....	12
2.2.4 Capability Comparison.....	13
2.3 Review of Relevant Technologies, Tools, and Frameworks.....	13
2.3.1 Programming Language and Runtime.....	13
2.3.2 Web and API Framework.....	13
2.3.3 LLM Orchestration and Provider.....	14
2.3.4 Observability and Telemetry Sources.....	14
2.3.5 Data Stores.....	14
2.3.6 Security and Deployment.....	14
2.3.7 Relevant Standards and Specifications.....	14
2.4 Gap Analysis.....	15
2.4.1 Grounding and response validation.....	15
2.4.2 Deployment control.....	15
2.4.3 Inspectable entry point.....	15
2.4.4 Recurring-condition acknowledgement.....	16
2.4.5 Operator approval for enforcement.....	16
2.4.6 Organisation-specific runbooks.....	16
2.4.7 Reproducibility.....	16
CHAPTER THREE: METHODOLOGY AND SYSTEM DESIGN.....	17
3.1 Project Methodology.....	17
3.1.1 Methodological Phases.....	17
3.1.2 Justification for the Methodology Choice.....	17

3.1.3	Tools Supporting the Methodology.....	18
3.2	Requirements Analysis.....	18
3.2.1	Functional Requirements.....	18
3.2.2	Non-Functional Requirements.....	19
3.2.3	Actors and Use Cases.....	20
3.3	System Architecture.....	21
3.3.1	Architectural Style.....	21
3.3.2	Components.....	21
3.3.3	Communication Patterns.....	22
3.3.4	High-Level Architecture.....	22
3.3.5	Deployment Topology.....	23
3.3.6	Heartbeat Verdict Pipeline.....	24
3.4	System Design.....	25
3.4.1	Database Design (Entity-Relationship Model).....	26
3.4.2	Use-Case Model.....	26
3.4.3	Sequence Diagram: One Heartbeat Cycle.....	27
3.4.4	Entity-Relationship Model.....	27
3.4.5	Verdict-Memory Design.....	28
3.4.6	State Machine: LangGraph Reasoning Agent.....	28
3.4.7	Workflow: Runbook RAG Indexing.....	29
3.5	Tools and Technologies Used.....	29
3.5.1	Backend and Orchestration.....	29
3.5.2	Security and Deployment.....	30
3.5.3	Telemetry Sources.....	30
CHAPTER FOUR: SYSTEM IMPLEMENTATION.....		31
4.1	Development Environment.....	31
4.1.1	Hardware and Operating System.....	31
4.1.2	Source Control and Continuous Integration.....	31
4.1.3	Editor and Tooling.....	31
4.2	Implementation Details.....	31
4.2.1	Repository Layout.....	31
4.2.2	The Reasoning Agent and LangGraph Loop.....	33
4.2.3	The Two LLM Reasoning Modes.....	34
4.2.4	Prompt Construction and Response Validation.....	34
4.2.5	The Verdict-Memory.....	34
4.2.6	Telemetry Collection Tools.....	35
4.2.7	The RAG Runbook Service.....	35
4.2.8	Threat Detection and CrowdSec Response.....	36
4.2.9	The API Service and Middleware.....	36
4.2.10	Persistence and Retention.....	36
4.2.11	The Dashboard.....	36
4.2.12	API Endpoints.....	38
4.3	Security, Performance, and Scalability Considerations.....	39
4.3.1	Authentication and Session Management.....	39
4.3.2	Rate Limiting and Output Hygiene.....	39
4.3.3	Auditability.....	39
4.3.4	Performance and Responsiveness.....	40
4.3.5	Scalability and Degradation.....	40
4.3.6	Deployment Boundary.....	40
4.4	Challenges Encountered.....	40

4.4.1 LLM Output Conformance.....	40
4.4.2 Scope Bounding and False Alarms.....	40
4.4.3 Signature Stability for the Memory.....	40
4.4.4 Security Defects Corrected During Review.....	41
4.4.5 Loki Query Semantics.....	41
CHAPTER FIVE: TESTING, RESULTS AND EVALUATION.....	42
5.1 Testing Strategy.....	42
5.1.1 Unit Tests.....	42
5.1.2 API and Persistence Tests.....	42
5.1.3 Acknowledgement Invariants.....	42
5.2 Automated Test Results.....	42
5.2.1 Coverage by Module.....	43
5.2.2 Representative Test Cases.....	44
5.3 Acknowledgement-Memory Evaluation.....	45
5.4 Limited Live Functional Verification.....	46
5.4.1 Runbook Retrieval Pilot.....	46
5.4.2 Practitioner Usability Evaluation Not Completed.....	46
5.5 Evaluation Against the Objectives.....	47
5.6 Discussion and Threats to Validity.....	47
5.6.1 Supported Findings.....	47
5.6.2 Limitations.....	47
5.6.3 Validity.....	48
CHAPTER SIX: SUMMARY, CONCLUSION AND RECOMMENDATIONS.....	48
6.1 Summary of the Project.....	48
6.2 Conclusion.....	49
6.3 Professional Contributions.....	49
6.3.1 Reproducible Reference Implementation.....	49
6.3.2 Acknowledgement Pattern.....	49
6.3.3 Documented Limits.....	49
6.4 Recommendations for Future Work.....	49
6.4.1 Evaluation.....	49
6.4.2 Test and Security Coverage.....	50
6.4.3 Architecture.....	50
REFERENCES.....	51
APPENDIX A: API ENDPOINT REFERENCE.....	54
A.1 Authentication and Health.....	54
A.2 Status, History, and Configuration.....	54
A.3 Verdict Memory.....	54
A.4 Threats.....	54
A.5 Runbooks (RAG).....	55
APPENDIX B: SAMPLE CODE SNIPPETS.....	55
B.1 Verdict Fingerprint and Signature Derivation.....	55
B.2 Severity Safeguard.....	56
B.3 Known-Conditions Memory Block.....	56
B.4 Deterministic Threat Detection.....	56
B.5 LangGraph Reasoning Loop.....	57
APPENDIX C: DOCKER COMPOSE CONFIGURATION.....	58
APPENDIX D: PROPOSED USABILITY EVALUATION INSTRUMENT.....	59
D.1 Task Script.....	59
D.2 System Usability Scale Questionnaire.....	59

LIST OF FIGURES

Figure 3.1: High-Level Architecture of InfraGuard AI.....	23
Figure 3.2: Deployed Single-VPS Topology for DevPlanner and InfraGuard AI.....	24
Figure 3.3: Heartbeat Pipeline and Remediation Boundary.....	25
Figure 3.4: InfraGuard AI Use-Case Model.....	26
Figure 3.5: Sequence of One Heartbeat Cycle.....	27
Figure 3.6: Verdict and Acknowledgement Data Model.....	28
Figure 3.7: LangGraph Agent State Machine.....	29
Figure 4.1: Repository Layout of the Reference Implementation.....	32
Figure 4.2: Verdict-Memory Fingerprint and Acknowledgement Flow.....	35
Figure 4.3: InfraGuard AI Operations Dashboard.....	37
Figure 4.4: Local Runbook Indexing and Retrieval Process.....	38
Figure 5.1: Selected Module Coverage from the Verification Run.....	44
Figure 5.2: Visible Duplicate Warnings With and Without Acknowledgement.....	45
Figure 5.3: Five-Case Practice Evaluation Workflow.....	46

LIST OF TABLES

Table 2.1: Comparative Review of Thirteen Studies in AIOps and LLM-Assisted Operations.	
.....	12
Table 2.2: Deployment and Scope Comparison of Representative Systems.....	13
Table 2.3: Relevant Standards and Specifications.....	15
Table 3.1: Functional Requirements of InfraGuard AI.....	19
Table 3.2: Non-Functional Requirements of InfraGuard AI.....	20
Table 3.3: Telemetry Sources and Collection Tools.....	30
Table 4.1: Repository Module Map and Responsibilities.....	33
Table 4.2: API Endpoint Reference.....	39
Table 5.1: Selected Module Coverage from the Verification Run (measured 16 August 2026).	
.....	43
Table 5.2: Representative Automated Test Cases and Results.....	45
Table 5.3: Objective and Evidence Matrix.....	47

CHAPTER ONE: INTRODUCTION

1.1 Background to the Project

Organisations increasingly operate containerised services that produce logs, metrics, and service events throughout the day. These signals are useful, but their volume and distribution make incident review difficult for a small operations team. An engineer may need to compare several sources before deciding whether a service is healthy, degraded, or failing.

Observability is the ability to infer a system's internal state from its outputs. Prometheus, Grafana Loki, OpenTelemetry, and related tools provide established ways to collect and inspect telemetry. They support detection, but they do not remove the need for a person to interpret the evidence and choose an appropriate response.

Site reliability engineering treats repetitive operational work as toil and recommends automation where the work is predictable and reviewable (Beyer et al., 2016). Initial incident triage fits this description in part: the inputs are available as telemetry, but a useful judgement still depends on context and experience. The purpose of this project is to assist that first review, not to replace the operator who remains responsible for action.

Large language models can summarise structured and unstructured technical material, and tool-using agents can retrieve additional evidence before producing an answer. These capabilities make LLM-assisted triage worth testing. They also create risks. A model may infer facts that are not present, misread an optional integration as a failure, or produce a response that does not match the expected structure. Prompt constraints, output validation, and human approval can reduce these risks, but they do not prove that every verdict is correct.

Nigerian organisations that operate digital services must also account for data-protection and sector-specific cybersecurity obligations. The Nigeria Data Protection Act 2023 sets duties for responsible processing of personal data. For institutions within its scope, the Central Bank of Nigeria's 2024 cybersecurity framework includes monitoring, reporting, and resilience controls (Central Bank of Nigeria, 2024; Federal Republic of Nigeria, 2023). Local deployment keeps the application records and access controls on the operator's host, but any prompt sent to a cloud LLM provider leaves that host.

LivWell (Integrated Wellness Inc.) provided the operational setting that informed the requirements: several environments, containerised workloads, a small team, and a need for clear incident records. These constraints led to a single-host design, bounded telemetry collection, request auditing, operator-approved threat response, and an acknowledgement feature for recurring conditions. The result is a working reference implementation called InfraGuard AI.

1.2 Statement of the Problem

Five practical issues remain even when monitoring tools are already in place:

1. Triage time. When several telemetry sources report a problem, a responder must compare the evidence before stating the likely cause and next action. This work tests whether a bounded first-pass summary can support that review.
2. Repeated low-risk findings. A known condition may appear on successive heartbeat cycles and occupy the same dashboard space as an unreviewed finding.
3. Ungrounded model output. An LLM can infer a cause that is not supported by the supplied telemetry or treat an optional integration as a fault. Its response therefore needs scope constraints, structured parsing, and operator review.
4. Deployment and data handling. Managed platforms and cloud model APIs differ in cost, hosting model, and the telemetry they receive. A small team needs a design whose data boundary can be inspected and configured.
5. Limited operator memory. A recurring condition may appear on every heartbeat even after an operator has reviewed it. Without an acknowledgement mechanism, the dashboard provides no durable way to distinguish a known low-risk condition from an unreviewed one.

The project therefore asks how a bounded, self-hosted triage system can turn available telemetry into a reviewable verdict, retrieve relevant runbooks, support operator-approved threat response, and remember recurring conditions without concealing high-risk findings.

1.3 Aim and Objectives of the Project

1.3.1 Aim

The aim of this research is to design, develop, and evaluate InfraGuard AI, an LLM-assisted observability and incident-triage system that produces structured verdicts from available telemetry, supports operator-approved threat response and runbook retrieval, and records acknowledgements for recurring low-risk conditions.

1.3.2 Specific Objectives

The study pursued five objectives:

1. To design and implement a multi-source telemetry-collection layer that uniformly gathers application and system logs (via Grafana Loki), infrastructure metrics (via Prometheus), endpoint health (via HTTP probes), and optional container state (via the Docker Engine API), and normalises them into a bounded context suitable for downstream LLM reasoning.

2. To develop a LangGraph reasoning workflow following a collect, analyse, decide, and notify cycle that requests a structured JSON verdict from a configured LLM provider and sends notifications for high and critical verdicts.
3. To design and implement a retrieval-augmented runbook assistant that indexes local Markdown procedures with local ONNX embeddings and ChromaDB, then supplies relevant excerpts to the selected language model.
4. To implement an acknowledgement memory keyed by a fingerprint of the condition signature, prompt version, and model, so matching low-risk verdicts can be marked and hidden by default while high and critical verdicts remain visible.
5. To package the system for single-host deployment through Docker Compose and evaluate the implemented behaviour through automated tests and measured code coverage. A practitioner usability protocol is also defined for later administration.

1.4 Scope of the Project

1.4.1 Within Scope

- Collection of application/system logs (Loki), infrastructure metrics (Prometheus), endpoint health (HTTP probes), and optional Docker container telemetry, on a configurable heartbeat interval.
- Model analysis through the Gemini Developer API, Anthropic, OpenAI-compatible services, or Ollama, selected through environment configuration.
- Push notification of high- and critical-severity verdicts via the ntfy.sh service.
- Deterministic detection of HTTP/SSH brute-force and port-scan patterns in logs, with operator-approved CrowdSec IP banning (operating in dry-run mode when CrowdSec is not configured).
- A local runbook retrieval path based on Markdown files, ONNX all-MiniLM-L6-v2 embeddings and a ChromaDB vector index.
- A stateful acknowledgement memory with condition-and-ruleset fingerprinting and a configurable time-to-live. A changed signature, prompt version, or model produces a different fingerprint.
- A single-page operations dashboard with session authentication, rate limiting, security headers, request auditing, and output escaping. Production deployment requires HTTPS termination and restricted network access.

1.4.2 Out of Scope (Identified as Future Work)

- Autonomous code remediation, including ingesting security-scanner output such as SARIF, generating code or configuration patches, and opening pull requests. The agent diagnoses and recommends; it does not modify application code.

- A comparative production evaluation of local and hosted models, including capacity planning for self-hosted inference, remains outside the implemented evaluation scope.
- Multi-server fleet operation. The present work targets a single host monitoring its co-located workloads; a distributed control-plane-and-satellite topology is future work.
- Custom training, fine-tuning, or distillation of the underlying LLMs. Models are treated as black-box providers selected through the common interface: hosted models are accessed through their APIs, while Ollama uses the local service.
- Identity and access-management federation with external single-sign-on providers; the system uses cookie-based session authentication only.
- Native mobile or desktop client applications; the dashboard is delivered exclusively as a responsive web application.
- Formal security certification (e.g. SOC 2 or an ISO 27001 audit). The deliverable is a reference implementation and architectural blueprint, not a certified product.

1.5 Significance of the Project

1.5.1 Significance to Industry

Practitioners can inspect how the implementation connects existing telemetry sources to an LLM-assisted review loop. The repository exposes the collection boundary, acknowledgement rule, threat-response approval step, and provider interface. It does not claim superiority over commercial platforms.

1.5.2 Significance to Organisations

A small organisation can deploy, test, and modify InfraGuard AI on one host without first adopting a large observability platform. Whether that arrangement is suitable depends on the organisation's scale and controls. Running cost also depends on the LLM provider, heartbeat interval, prompt size, and surrounding infrastructure.

1.5.3 Significance to Policy and Society

The Nigeria Data Protection Act 2023 and the Central Bank of Nigeria cybersecurity framework are relevant to organisations handling personal or financial data (Central Bank of Nigeria, 2024; Federal Republic of Nigeria, 2023). InfraGuard AI records verdicts, acknowledgements, and API requests, which an organisation may retain as part of its monitoring records. These records do not prove compliance. Use of a cloud LLM also requires an appropriate data-protection assessment.

1.5.4 Significance to Professional Practice

Students and practitioners can trace a complete path from telemetry adapters through the LLM workflow, API, persistence layer, dashboard, and tests. The report distinguishes the

limited deployed runbook pilot from the controlled live diagnostic and practitioner evaluations that remain outstanding.

1.5.5 Significance to the Master of Information Technology Programme

For the Master of Information Technology programme, the work links an operational problem to a software artefact, automated tests, measured coverage, and stated limits. The report distinguishes completed verification from evaluation proposed for later work.

CHAPTER TWO: LITERATURE REVIEW AND TECHNOLOGY CONTEXT

2.1 Conceptual Review

The design draws on six concepts: cloud-native observability, site reliability engineering, AIOps, large language models and agents, retrieval-augmented generation, and acknowledgement memory. Each one influenced a specific part of the implementation.

2.1.1 Cloud-Native Observability and the Three Pillars

Observability is the ability to infer a system's internal state from its outputs. Common telemetry types include metrics, logs, traces, and service events. Prometheus is widely used for metrics, Grafana Loki for logs, and OpenTelemetry for instrumentation. These tools make signals available for investigation, but an operator must still determine what the signals mean in the context of a particular service. InfraGuard AI consumes logs and metrics directly, supplemented by HTTP probes and optional Docker telemetry.

2.1.2 Site Reliability Engineering and Incident Management

Site reliability engineering uses telemetry, service-level objectives, and operational automation to improve reliability. Beyer et al. (2016) use the term toil for repetitive manual work that scales with service growth. Alert review and repeated handling of known conditions are relevant examples. InfraGuard AI addresses a limited part of this work by generating a first-pass verdict and recording operator acknowledgements.

2.1.3 AIOps: Artificial Intelligence for IT Operations

AIOps applies machine learning and analytics to operations data for tasks such as anomaly detection, event correlation, fault diagnosis, and remediation support. Chen et al. (2025) present AIOpsLab as a reproducible environment for deploying cloud systems, injecting faults, exporting telemetry, and evaluating LLM agents across operational tasks. Their results also show that agent capability depends on the task and evaluation setting. InfraGuard AI does not train an anomaly detector or claim autonomous remediation. It asks a general-purpose LLM to interpret a bounded set of collected evidence and returns the result for operator review.

2.1.4 Large Language Models and Agentic AI

Large language models generate text from a supplied context and can be connected to tools that retrieve additional evidence (Gemini 2.5 Team, 2025). ReAct interleaves model reasoning with tool actions (Yao et al., 2023), while Reflexion studies the use of verbal feedback across attempts (Shinn et al., 2023). InfraGuard AI uses LangGraph to define a

bounded workflow and offers an optional LangChain mode in which the configured model selects from a small telemetry tool set (LangChain, n.d.). These mechanisms do not guarantee a correct diagnosis, so the system keeps the operator responsible for action.

2.1.5 Retrieval-Augmented Generation

The retrieval-augmented generation layer uses local Markdown runbooks as its knowledge source (Lewis et al., 2020). Files are loaded from the read-only runbook mount, divided into retrievable documents, embedded locally with the ONNX all-MiniLM-L6-v2 model and stored in ChromaDB. For each operator question, the four most relevant excerpts are supplied to the selected model. This keeps the source material inspectable and avoids dependence on an external documentation service.

2.1.6 Statelessness, Alert Deduplication, and Agent Memory

A scheduled triage loop can show the same condition on successive cycles even after it has been reviewed. Alert grouping and log-template research show why the identity used for matching matters. A broad key can merge different conditions, while exact raw text often fails to match because timestamps, identifiers, and values change. Drain, for example, derives stable log templates by separating fixed and variable tokens (He et al., 2017). InfraGuard AI applies a related idea at verdict level: it combines a condition signature with the prompt version and model to produce a fingerprint. A matching acknowledgement marks an ok or warning verdict as suppressed in the API and dashboard. It does not prevent the model call on the next heartbeat.

2.2 Review of Existing Systems and Solutions

Thirteen studies were reviewed across site reliability engineering, AIOps, log parsing, retrieval, and LLM-assisted operations. They are compared by method, evidence, limitations, and relevance to InfraGuard AI. Product documentation is considered separately in the discussion of commercial and open-source tools.

2.2.1 Comparative Table of Thirteen Recent and Related Studies

Table 2.1 brings together thirteen studies on site reliability engineering, AIOps, LLM-based incident analysis, log parsing, tool use, and retrieval. The selection favours work published from 2020 to 2025, established research venues, and studies that bear directly on the implemented design.

#	Study (author, year)	Method	Tools / models	Reported result	Limitation	Use in InfraGuard AI
1	Beyer, Jones, Petoﬀ & Murphy (2016). “Site Reliability Engineering.” O’Reilly / Google.	Foundational practitioner text codifying SRE practice: service-level objectives, error budgets, on-call, and incident response.	Operational framework; no software artefact.	Defines telemetry-driven operations and describes repetitive manual work that grows with service scale as toil.	Predates the LLM era; assumes a human performs the interpretation of telemetry into action.	Implements a first-pass telemetry summary as a structured verdict. Its effect on operator workload still requires a field study.
2	Chen, Shetty, Somashekar et al. (2025). “AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds.” MLSys.	Builds a framework that deploys cloud environments , injects faults, generates workloads, exports telemetry, and evaluates operational agents.	Open AIOpsLab framework, agent-cloud interface, benchmark environments, and LLM agents.	Provides a reproducible benchmark and reports that current agents have uneven capability across complex operational tasks.	Focuses on benchmarked agent evaluation rather than a small operator-facing deployment with persistent acknowledgement memory.	Motivates reproducible evaluation while InfraGuard AI supplies an inspectable single-host implementation and states which live-model tests remain outstanding.
3	He, Zhu, Zheng & Lyu (2017). “Drain: An Online Log Parsing Approach with Fixed Depth Tree.” IEEE ICWS.	Streaming log parser that abstracts raw log lines into templates by stripping variable tokens via a fixed-depth parse tree.	The Drain algorithm.	Enables stable grouping of semantically identical log events despite volatile parameters such as timestamps and identifiers.	Provides parsing only; no reasoning over the parsed events and no operator-facing memory of judgements.	The verdict-memory applies the same volatile-token-normalisation principle so that the same condition fingerprints stably across heartbeats, enabling reliable acknowledgement.

#	Study (author, year)	Method	Tools / models	Reported result	Limitation	Use in InfraGuard AI
4	Yao, Zhao, Yu, Du, Shafran, Narasimhan & Cao (2023). “ReAct: Synergizing Reasoning and Acting in Language Models.” ICLR.	Introduces an interleaved reasoning-and-acting prompting paradigm allowing an LLM to alternate chain-of-thought with tool invocations.	GPT-3 / PaLM with external tool environments.	Outperforms the reported reasoning-only and acting-only baselines on the study’s knowledge and decision tasks.	Domain-agnostic foundational technique with no operations application or production evaluation.	Adopted in the LangChain multi-tool mode, where Gemini interleaves reasoning with calls to the Loki, Prometheus, and HTTP-probe tools before producing a verdict.
5	Shinn, Cassano, Gopinath, Narasimhan & Yao (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning.” NeurIPS.	Self-improving agent loop in which the model reflects on failed actions through verbal critique and revises its plan.	GPT-4 with a reflection memory.	Outperforms the reported single-shot baselines on the study’s multi-step tasks.	Demonstrated on toy code and QA tasks; no integration with telemetry or operations systems.	Provides a contrast with InfraGuard AI: the project stores operator acknowledgements, not a model-generated reflection memory.
6	Yuan, Tang, Liu et al. (2025). “Incident Diagnosing and Reporting System Based on Retrieval Augmented Large Language Model.” AAAI.	RAIDR retrieves system documents and similar incident records, then uses an LLM to diagnose IoT anomalies and draft incident reports.	Incident signatures, relationship-aware anomaly analysis, retrieval, and an LLM report generator.	Demonstrates a retrieval-augmented incident diagnosis and reporting workflow over IoT sensor data.	Short demonstration paper focused on IoT incidents; it does not evaluate the single-host observability and acknowledgment workflow used here.	Supports the use of organisation-specific documents for incident assistance; InfraGuard AI applies retrieval to runbooks and returns source titles.

#	Study (author, year)	Method	Tools / models	Reported result	Limitation	Use in InfraGuard AI
7	Ahmed, Ghosh, Bansal, Zimmermann, Zhang and Rajmohan (2023). "Recommending Root-Cause and Mitigation Steps for Cloud Incidents Using Large Language Models." ICSE.	Prompts and fine-tunes LLMs to recommend root cause and mitigation steps from production incident data at scale.	GPT-3.0 and GPT-3.5 models on more than 40,000 Microsoft incidents.	Reports zero-shot, fine-tuned, and multi-task results, followed by evaluation with incident owners.	Operates inside a proprietary hyperscale cloud; not self-hostable; no operator-facing memory; no open implementation.	Brings grounded root-cause and recommended-action verdicts to a self-hosted, open reference system sized for small teams, with an operator acknowledgment memory layered on top.
8	Jin, Zhang, Ma et al. (2023). "Assess and Summarize: Improve Outage Understanding with Large Language Models." ESEC/FSE.	Oasis groups incidents to assess outage impact and uses a fine-tuned model to generate an outage summary.	Fine-tuned GPT-3.x models on data from 18 Microsoft cloud systems.	Reports an empirical evaluation and a human evaluation with outage owners; a prototype entered experimental adoption.	Evaluates outage summarisation after evidence has been collected; the data and deployment are proprietary.	Runs on a scheduled heartbeat and stores each structured verdict for later review and acknowledgment.
9	Chen, Xie, Ma et al. (2024). "Automatic Root Cause Analysis via Large Language Models for Cloud Incidents." EuroSys.	RCACopilot matches incidents to handlers by alert type, gathers diagnostic information, predicts a root-cause category, and generates an explanation.	Incident matching and diagnostic collection with an LLM-generated explanation over Microsoft incident data.	Reports root-cause accuracy up to 0.766; its diagnostic collection component had been used at Microsoft for more than four years.	Targets hyperscale operations; heavyweight; not bounded for, or deployable by, small single-host estates.	Restricts collection to selected sources on one host and exposes the implementation for inspection.

#	Study (author, year)	Method	Tools / models	Reported result	Limitation	Use in InfraGuard AI
10	Roy, Zhang, Bhawe et al. (2024). “Exploring LLM-Based Agents for Root Cause Analysis.” FSE Companion.	Investigates autonomous LLM agents for root-cause analysis and the interface and grounding choices that make them effective.	Tool-augmented LLM agents on incident datasets.	Reports that tool and grounding choices affect root-cause-analysis performance.	Uses Microsoft incident data and diagnostic services; it does not present a general self-hosted deployment or an acknowledgment feature.	Contributes the self-hosted, human-in-the-loop, memory-equipped variant: operator-approved threat response and a stateful acknowledgment memory.
11	Xu, Zhang, Zhong et al. (2025). “OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?” ICLR.	Introduces a benchmark for locating software-failure root causes from heterogeneous logs, metrics, traces, and dependency information.	OpenRCA: 335 failures from three enterprise systems and more than 68 GB of telemetry; evaluated LLMs and an RCA agent.	The best reported configuration, an RCA agent using Claude 3.5, solved 11.34 percent of the benchmark cases.	Offline benchmark rather than a deployed incident workflow; results show that current models remain unreliable on difficult cases.	Supports the project’s human-review boundary and the decision not to claim diagnostic accuracy from software tests alone.
12	Hou, Zhao, Liu et al. (2024). “Large Language Models for Software Engineering: A Systematic Literature Review.” ACM TOSEM.	Systematic review of 395 studies on LLM applications across the software lifecycle.	Meta-analysis.	Catalogues models, data practices, evaluation methods, and software-engineering tasks, and identifies uneven coverage across the field.	Reviews the gap but does not provide an operations implementation ; it also identifies hallucination and reproducibility concerns.	Provides an inspectable implementation of one LLM-assisted operations workflow and documents its output constraints.
13	Guo, Yang, Lu et al. (2024). “OWL: A Large Language Model for IT Operations.” ICLR.	Develops a domain-specialised LLM and benchmark for IT-operations question answering.	Domain-tuned LLM; IT-ops benchmark.	Domain specialisation improves operations question-answering performance over general models.	Model-centric; no end-to-end agent, no live telemetry loop, and the bespoke model raises cost and operational complexity.	Uses a configurable general-purpose provider with bounded telemetry context instead of training a domain-specific model.

Table 2.1: Comparative Review of Thirteen Studies in AIOps and LLM-Assisted Operations.

2.2.2 Synthesis of the Reviewed Studies

Ahmed et al. (2023), Jin et al. (2023), Chen et al. (2024), and Roy et al. (2024) evaluate LLMs for incident recommendations, outage summaries, or tool-assisted root-cause analysis, largely with proprietary cloud data. More recent open work addresses reproducible evaluation and exposes the remaining difficulty: AIOpsLab evaluates agents in controlled cloud environments, while OpenRCA reports that its best configuration solved 11.34 percent of 335 benchmark failures (Chen et al., 2025; Xu et al., 2025). RAIDR applies retrieval to incident diagnosis and reporting in an IoT setting (Yuan et al., 2025). InfraGuard AI uses related tool and retrieval patterns over a small telemetry surface, but keeps the operator responsible for action and does not claim diagnostic accuracy from software tests.

2.2.3 Commercial and Open-Source State of the Art

Four products provide useful points of comparison. The descriptions below are limited to capabilities stated in their current product documentation.

Datadog Bits AI and Watchdog

Datadog Bits AI supports investigations using Datadog telemetry, runbooks, environment context, feedback, and investigation memories (Datadog, n.d.). It is part of Datadog's managed commercial platform.

New Relic AI

New Relic AI provides natural-language access to telemetry held in the New Relic platform and can return explanations, summaries, charts, and troubleshooting guidance (New Relic, n.d.). It is a managed platform feature rather than an operator-hosted reference implementation.

PagerDuty AIOps

PagerDuty AIOps provides alert grouping, noise reduction, triage, root-cause features, event orchestration, and automation within PagerDuty's managed incident platform (PagerDuty, n.d.). Its event-grouping functions are related to, but not identical with, the acknowledgement memory studied here.

k8sgpt (Open Source)

k8sgpt is an open-source tool that analyses Kubernetes resources and sends selected information to a configured AI backend for explanation (k8sgpt, n.d.). Its primary scope is Kubernetes diagnosis rather than the multi-source heartbeat implemented by InfraGuard AI.

2.2.4 Capability Comparison

Table 2.2 compares the reviewed systems by their main emphasis and deployment model. Binary capability scores were avoided because commercial features change and cannot be fully verified from public documentation.

System	Deployment	Primary Scope	Relevant Distinction
InfraGuard AI	Operator-hosted reference implementation	Scheduled triage over selected telemetry sources	Condition-and-ruleset acknowledgement; human-approved CrowdSec action
Datadog Bits AI	Managed commercial platform	Investigation over Datadog telemetry and knowledge sources	Runbooks, environment context, feedback, and investigation memories
New Relic AI	Managed commercial platform	Natural-language analysis of New Relic telemetry	Platform-integrated explanations, queries, summaries, and charts
PagerDuty AIOps	Managed commercial platform	Event grouping, triage, RCA support, and orchestration	Noise reduction and event-processing automation
k8sgpt	Open-source tool run by the user	Kubernetes resource analysis	Kubernetes-specific analyzers with a selectable AI backend

Table 2.2: Deployment and Scope Comparison of Representative Systems.

2.3 Review of Relevant Technologies, Tools, and Frameworks

Technology choices were considered by architectural layer, with attention to maintenance status, licensing, self-hosting support, and suitability for a small single-host deployment.

2.3.1 Programming Language and Runtime

The implementation uses Python with FastAPI, LangGraph, LangChain, HTTPX, ChromaDB, ONNX-based sentence embeddings, Docker SDK integrations and provider-specific clients.

2.3.2 Web and API Framework

FastAPI supports the asynchronous routes used by the application and generates an OpenAPI description of the API. Its dependency mechanism is used for route-level controls. The same application serves the dashboard and its JSON endpoints, with both using the session cookie for authentication. This avoids a separate dashboard service.

2.3.3 LLM Orchestration and Provider

Provider dispatch is controlled by LLM_PROVIDER. The default path uses the Gemini Developer API with gemini-3.6-flash; Anthropic, OpenAI-compatible endpoints and Ollama are also implemented. The provider and model name form part of the acknowledgement fingerprint, so a model change forces a fresh review of an otherwise recurring condition.

2.3.4 Observability and Telemetry Sources

Prometheus and Grafana Loki are the primary metric and log sources and are queried through their HTTP APIs. The deployment expects those services to be configured separately; it does not deploy a log shipper or monitoring backend. HTTP probes provide endpoint checks, the Docker Engine API provides optional local-container telemetry, and CrowdSec is available for operator-approved IP bans. Distributed tracing is outside the project scope.

2.3.5 Data Stores

Runbook embeddings are generated locally with the ONNX all-MiniLM-L6-v2 model. The vectors and document metadata are stored in ChromaDB, allowing retrieval without sending the runbook corpus to an embedding API.

2.3.6 Security and Deployment

Application controls include signed timestamped session cookies, rate limiting, response headers, output escaping, and request logging. Docker Compose packages the API and agent, Terraform provisions a Google Compute Engine host, and GitHub Actions runs tests before the deployment job builds and publishes images. A production deployment still requires HTTPS termination, restricted firewall rules, and protected secret handling.

2.3.7 Relevant Standards and Specifications

Table 2.3 lists the standards and specifications referenced in the design.

Standard / Specification	Source	Role in InfraGuard AI
Prometheus Exposition Format	Prometheus Authors (n.d.)	Metric data format consumed by the metrics-collection tool.
Loki HTTP API (LogQL)	Grafana Labs (n.d.)	Log query interface used to retrieve recent log lines over an explicit time window.
OpenAPI 3.1	OpenAPI Initiative (2025)	Automatic description of the control-plane REST surface generated by FastAPI.
System Usability Scale (SUS)	Brooke (1996)	Proposed standardised instrument for a later practitioner study; it was not administered in this evaluation.

Standard / Specification	Source	Role in InfraGuard AI
ISO/IEC 27001:2022	International Organization for Standardization (2022)	Information-security management reference informing audit-trail and access-control design.
Nigeria Data Protection Act 2023	Federal Republic of Nigeria (2023)	Regional legal context for access control, accountability, and responsible processing.
Risk-Based Cybersecurity Framework for DMBs and PSBs	Central Bank of Nigeria (2024)	Sector-specific context for monitoring, reporting, and resilience controls.
NIST AI 600-1	National Institute of Standards and Technology (2024)	Generative-AI risk reference for grounding, output validation, human oversight, and post-deployment monitoring.
OWASP Top Ten	OWASP Foundation (2025)	Reference for the dashboard's security-header, session, and threat-handling choices.

Table 2.3: Relevant Standards and Specifications.

2.4 Gap Analysis

The review exposed seven gaps that informed the system design.

2.4.1 Grounding and response validation

An LLM can infer an unsupported cause or return malformed output. The NIST Generative AI Profile identifies confabulation, information security, human oversight, and post-deployment monitoring as risk-management concerns (National Institute of Standards and Technology, 2024). InfraGuard AI supplies selected telemetry, tells the model not to treat absent integrations as faults, and checks the response fields and severity. These controls constrain the interface, but they do not establish diagnostic accuracy.

2.4.2 Deployment control

The commercial platforms reviewed are vendor-managed services. InfraGuard AI keeps the API, dashboard, database, and runbook index on the operator's host. A prompt still leaves the host when the operator selects a cloud LLM provider.

2.4.3 Inspectable entry point

A small reference implementation allows the collection and reasoning paths to be inspected without adopting a full observability platform. This does not imply lower operating cost; cost depends on the host, provider, prompt size, and request rate.

2.4.4 Recurring-condition acknowledgement

A scheduled triage loop needs a durable distinction between a reviewed low-risk condition and a new finding. InfraGuard AI stores a fingerprinted acknowledgement and suppresses the matching ok or warning verdict in the default view. A change to the signature, prompt version, or model produces a different fingerprint.

2.4.5 Operator approval for enforcement

IP bans can interrupt legitimate traffic. InfraGuard AI therefore applies a CrowdSec ban only after an operator approves the detected threat. Acknowledgement rules also prevent high and critical findings from being hidden.

2.4.6 Organisation-specific runbooks

Operational procedures are maintained as local Markdown files under the runbooks directory. Docker Compose mounts this directory read-only into the API and agent containers, while subdirectories provide category metadata for retrieval.

2.4.7 Reproducibility

Private datasets and managed infrastructure limit direct reproduction of several reviewed systems. InfraGuard AI keeps the collection, matching, persistence, and API logic in one repository so another evaluator can inspect those decisions.

These gaps set the boundary for the implementation. Chapter Three translates them into requirements and design decisions.

CHAPTER THREE: METHODOLOGY AND SYSTEM DESIGN

3.1 Project Methodology

An iterative design-science method guided the project. Design-science research suits work that builds and evaluates a software artefact in response to a defined operational problem (Hevner et al., 2004). Short implementation cycles allowed the requirements, code, tests, and documentation to change together during the build-and-evaluate process.

3.1.1 Methodological Phases

The work moved through six overlapping phases. Test results or code review sometimes required an earlier decision to be revisited.

1. Problem identification. Reviewed observability, SRE, AIOps, incident-analysis, RAG, and agent research, then defined the narrow problem of first-pass telemetry interpretation for a small team.
2. Objective definition. Converted the problem into five implementation and evaluation objectives.
3. Design and development. Implemented telemetry adapters, the reasoning workflow, provider abstraction, runbook retrieval, threat handling, persistence, API, dashboard, and acknowledgement memory.
4. Demonstration. Exercised the main workflow paths with mocked upstream services and controlled test fixtures.
5. Evaluation. Ran the automated test and coverage suite, compared the displayed duplicate count before and after an acknowledgement, and completed a limited deployed runbook-retrieval pilot. The practitioner usability instrument was prepared but not administered.
6. Communication. Updated the report, diagrams, references, and appendices so that they describe the implemented system and measured evidence.

3.1.2 Justification for the Methodology Choice

A strictly sequential method would have been a poor fit because several requirements became clearer only during implementation, particularly output parsing, acknowledgement matching, and middleware order. Design science keeps the problem, artefact, and evaluation connected while leaving room to revise those details.

3.1.3 Tools Supporting the Methodology

The Git repository contains the source code, infrastructure definitions, tests, and report sources. Two GitHub Actions workflows are included: pull requests run the test suite, while pushes to the main branch run tests before the container build and deployment steps.

3.2 Requirements Analysis

Requirements came from the operational setting and the agreed project scope. Each one was checked against the repository and classified as Must or Should, depending on whether it belonged to the core heartbeat, an operator control, or an optional integration. Work outside the delivered boundary remains listed as future work in Section 1.4.2.

3.2.1 Functional Requirements

ID	Requirement	Description	Priority
FR-01	Telemetry Collection	The system shall collect logs (Loki), metrics (Prometheus), endpoint health (HTTP probes), and optional container state (Docker Engine API) on a configurable heartbeat interval.	Must
FR-02	Structured Verdict Generation	The system shall send collected context to the configured LLM provider and request a verdict containing severity, summary, root cause, recommended action, and signature.	Must
FR-03	Severity Classification	Each verdict shall be classified into one of four severities: ok, warning, high, or critical.	Must
FR-04	Dual Reasoning Modes	Support both direct provider calls and LangChain multi-tool reasoning mode.	Must
FR-05	Notification	The system shall dispatch a push notification via ntfy.sh for verdicts of high or critical severity.	Must
FR-06	Verdict Persistence and Retention	Verdicts shall be persisted to SQLite and automatically pruned after a configurable retention window.	Must
FR-07	Verdict Memory (Acknowledgement)	The system shall fingerprint each verdict on its condition signature plus ruleset version and allow an operator to acknowledge a low-risk condition so that a matching verdict is marked suppressed.	Must
FR-08	Memory Matching	A changed condition signature, prompt version, or model shall produce a different fingerprint; acknowledgements may also expire after a configurable time-to-live.	Must
FR-09	Severity Safeguard	High- and critical-severity verdicts shall never be suppressed by an acknowledgement.	Must

ID	Requirement	Description	Priority
FR-10	Threat Detection	The system shall scan recent logs for HTTP/SSH brute-force and port-scan patterns and surface detected threats with their source IPs.	Must
FR-11	Operator-Approved Threat Response	On explicit operator approval, the system shall build and apply a CrowdSec ban for a detected threat, operating in dry-run mode when CrowdSec is unconfigured.	Should
FR-12	Runbook Indexing	Index and retrieve local Markdown runbooks using local embeddings and ChromaDB.	Must
FR-13	Runbook Query (RAG)	The system shall answer operator questions over the indexed runbooks, returning a grounded answer and source citations.	Must
FR-14	Operations Dashboard	The system shall provide a web dashboard exposing current status, integration configuration, check history, the threat panel, and the runbook assistant.	Must
FR-15	Integration Status	The system shall report which integrations are configured, exposing booleans only and never secret values.	Should

Table 3.1: Functional Requirements of InfraGuard AI.

3.2.2 Non-Functional Requirements

ID	Quality Attribute	Requirement	Target
NFR-01	Cost Control	The heartbeat interval and prompt size shall be configurable so provider usage can be bounded.	One analysis request per heartbeat
NFR-02	Grounding Controls	The prompt shall include only configured evidence sections and shall instruct the model not to treat absent integrations as failures.	Covered by prompt-construction tests
NFR-03	Responsiveness	Runbook indexing and embeddings shall remain local to the deployment.	No direct blocking call in route coroutine
NFR-04	Authentication	All routes other than /health and /login shall require a valid signed session cookie.	24-hour maximum age
NFR-05	Rate Limiting	The API shall enforce request limits, with a stricter limit on the login route.	60/min; 5/min on login
NFR-06	Output Hygiene	Server-derived text rendered in the dashboard shall be HTML-escaped and response security headers shall be applied.	Escaping and headers covered by tests

ID	Quality Attribute	Requirement	Target
NFR-07	Auditability	Each request shall be logged with timestamp, IP, route, method, user, status, and latency.	Structured local request log
NFR-08	Portability	The API and agent shall deploy through Docker Compose on one Linux host.	Single-host composition
NFR-09	Maintainability	The backend shall have an automated suite with measured line coverage.	Coverage reported, not used as a sole quality measure
NFR-10	Data Minimisation	Only selected telemetry sections shall be included in the prompt sent to the configured LLM provider.	No full observability-store export
NFR-11	Storage Retention	Verdict storage shall be bounded by automatic retention pruning.	Default 30 days, configurable
NFR-12	Signature Stability	Fallback signature derivation shall remove volatile IP, hash, and numeric tokens.	Deterministic tests

Table 3.2: Non-Functional Requirements of InfraGuard AI.

3.2.3 Actors and Use Cases

Three actors appear in the use-case model. The Operator is responsible for the monitored infrastructure; the InfraGuard Agent runs each heartbeat without direct input; and the configured LLM Provider supplies the external model service. Their interactions are shown in Figure 3.4.

- UC-01: View the current status and verdict (Operator → System).
- UC-02: Inspect the root cause and recommended action of a verdict (Operator → System).
- UC-03: Acknowledge a verdict as a known non-issue (Operator → System).
- UC-04: Un-acknowledge a condition so it re-opens (Operator → System).
- UC-05: Review detected threats (Operator → System).
- UC-06: Apply a CrowdSec ban for a detected threat (Operator → System).
- UC-07: Ask the runbook assistant a question (Operator → System).
- UC-08 — Consult a Local Runbook: the operator submits an incident question, the application retrieves relevant excerpts from the indexed Markdown corpus and the selected model produces an answer grounded in those excerpts.
- UC-09: Generate a verdict on a heartbeat, consulting the LLM provider (Agent → LLM Provider).
- UC-10: Dispatch a notification on a high or critical verdict (Agent → Operator).

3.3 System Architecture

InfraGuard AI runs as two processes. The agent process executes the reasoning heartbeat, while the API process serves the dashboard and REST surface. Both share the same SQLite database. Long-running integration calls are moved to worker threads so they do not block the API event loop. Figure 3.1 shows the main components.

3.3.1 Architectural Style

The single-host architecture keeps collection, verdict generation, and memory logic separate from the Loki, Prometheus, HTTP-probe, Docker, and LLM-provider adapters. Each adapter has a small interface, which makes a source easier to omit, replace, or test independently.

3.3.2 Components

1. Agent (Heartbeat). A long-running asyncio process that, on each interval, fetches operator acknowledgements, collects telemetry, invokes the LangGraph reasoning loop, persists the verdict, and dispatches notifications. Implemented in `agent/main.py` and `agent/orchestrator.py`.
2. FastAPI Service. Hosts the dashboard and the REST API (status, alerts, configuration, threats, runbooks, and acknowledgement endpoints) behind authentication, rate-limiting, security-header, and audit middleware. Implemented in `api/main.py`.
3. Reasoning Substrate. A LangGraph state machine implements the collect-analyse-decide-notify loop with two interchangeable analysis modes: a single structured provider call and a LangChain multi-tool agent.
4. LLM Client. A provider layer supporting the Gemini Developer API, Anthropic, OpenAI, and optional Ollama models through a common verdict schema.
5. Verdict-Memory. The fingerprinting and acknowledgement subsystem (`agent/memory.py` and the acknowledgements table) that suppresses already-triaged conditions and feeds them back into the prompt as known conditions.
6. RAG Service. A retrieval pipeline over local Markdown runbooks, local ONNX all-MiniLM-L6-v2 embeddings, and a ChromaDB vector store.
7. Threat Subsystem. A deterministic detector that scans recent logs for attack patterns and an operator-approved CrowdSec adapter.
8. SQLite Store. The persistence layer for verdicts and acknowledgements, with automatic retention pruning.
9. Dashboard. A single self-contained HTML, CSS, and JavaScript page served as a static asset by FastAPI and communicating over JSON.
10. External Services. The selected cloud-model API (Gemini, Anthropic, or OpenAI), ntfy.sh, and the optional CrowdSec Local API. Ollama provides an optional local-model path.

3.3.3 Communication Patterns

During each heartbeat, the agent queries the Loki and Prometheus HTTP APIs, runs the configured HTTP probes, and may read from the Docker Engine API. Verdicts and acknowledgements are stored in the SQLite database shared with the API. The dashboard exchanges JSON with the API and uses a session cookie for authentication. HTTPS termination is left to the deployment environment.

3.3.4 High-Level Architecture

Figure 3.1 presents the high-level architecture. The scheduled agent and FastAPI service share the SQLite verdict store, while the API also uses the local ChromaDB index and a read-only mount containing twenty Markdown runbooks. The agent reads configured telemetry sources and calls the selected provider. It sends ntfy alerts for high or critical verdicts. The API calls the CrowdSec Local API only after an operator approves an IP ban; other remediation remains manual.

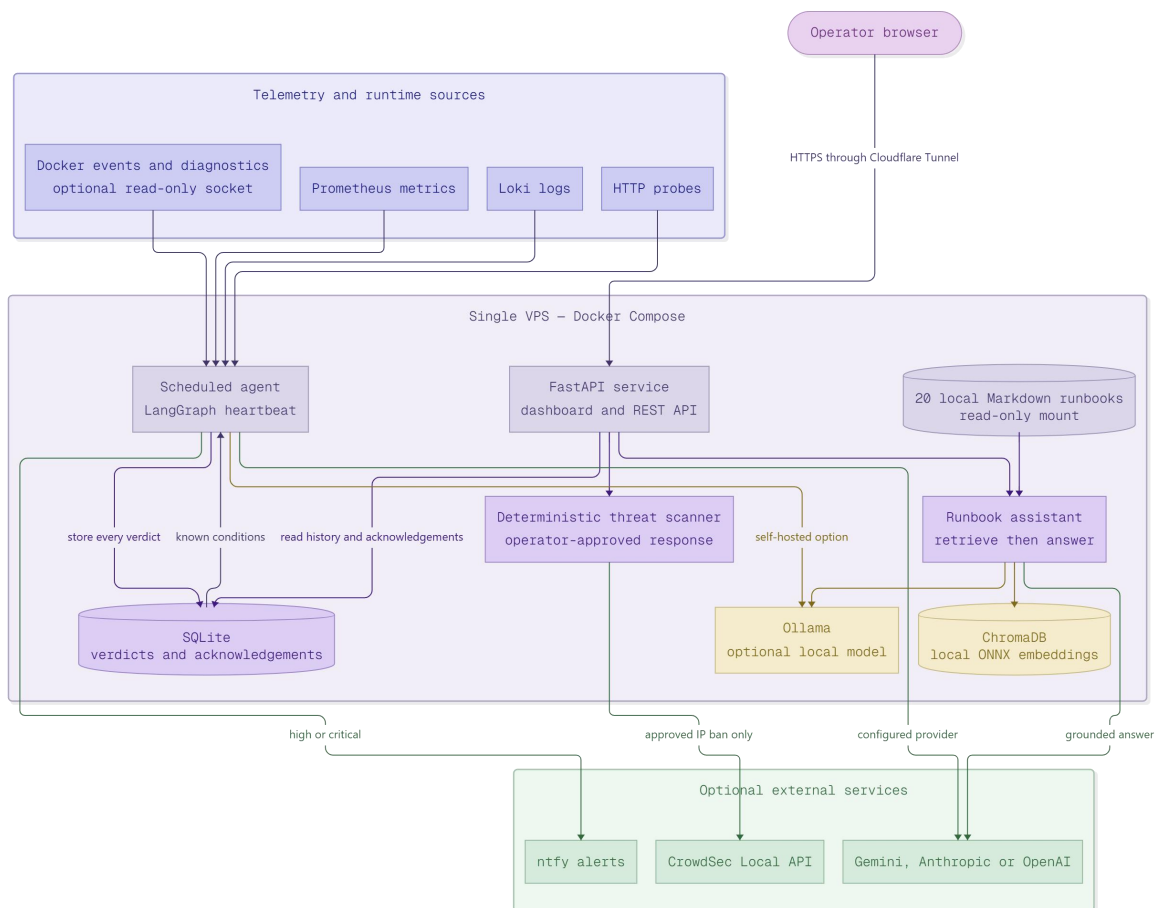


Figure 3.1: High-Level Architecture of InfraGuard AI.

3.3.5 Deployment Topology

Figure 3.2 depicts the deployed single-VPS topology. DevPlanner and InfraGuard AI run as separate Docker Compose projects on the same Linux host. Cloudflare Tunnel provides the public HTTPS entry point, while InfraGuard reaches its API, Loki, Prometheus, and CrowdSec through loopback-bound origins. Runbooks and application state remain on local volumes. Only the selected cloud-model API and ntfy require outbound access; Ollama can be used for local inference.

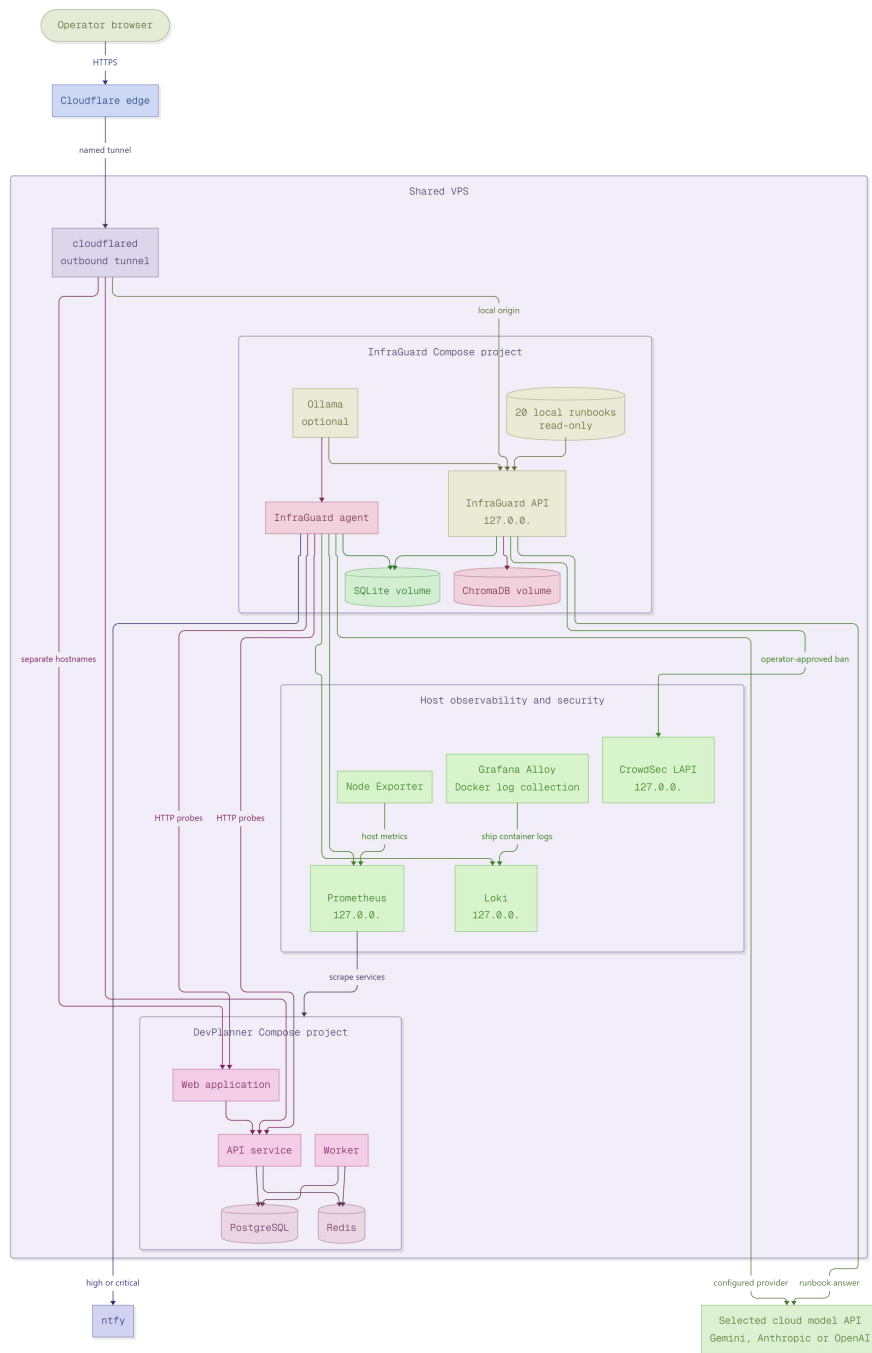


Figure 3.2: Deployed Single-VPS Topology for DevPlanner and InfraGuard AI.

3.3.6 Heartbeat Verdict Pipeline

Figure 3.3 traces the heartbeat and remediation boundary. The agent loads active acknowledgements, collects configured telemetry, calls the selected model, validates the structured verdict, computes its fingerprint, and stores it. A high or critical verdict may add Docker diagnostics and trigger ntfy. Separately, deterministic log rules may identify a threat, but an IP ban is submitted to CrowdSec only after operator approval. The LLM does not restart services, delete data, change configuration, or alter the firewall.

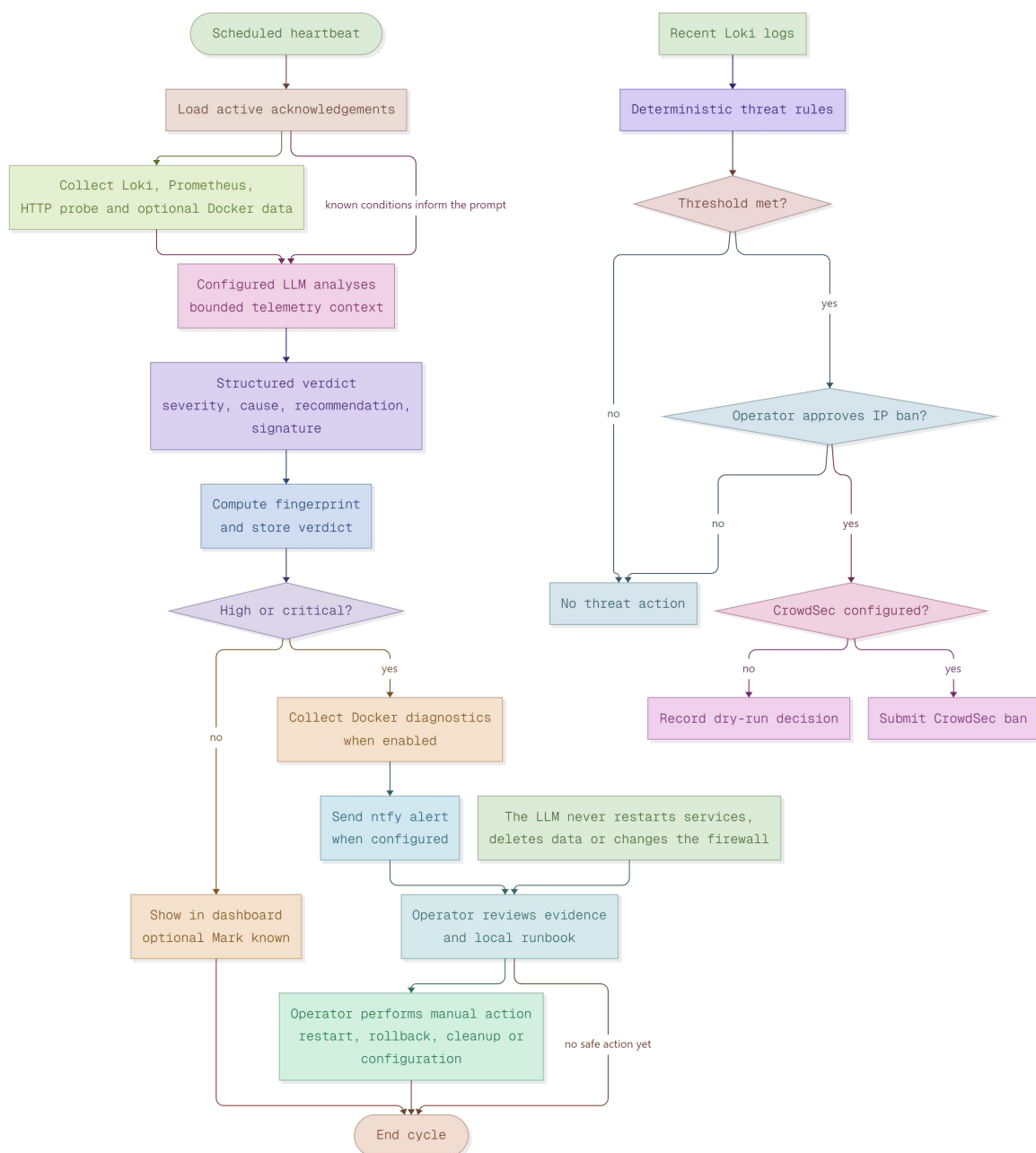


Figure 3.3: Heartbeat Pipeline and Remediation Boundary.

3.4 System Design

The design work focused on five areas: the database schema, the normal heartbeat sequence, acknowledgement matching, the reasoning state machine, and runbook indexing.

3.4.1 Database Design (Entity-Relationship Model)

Operational state is kept in two SQLite tables. ChromaDB holds the runbook embeddings, while API audit events are written to an append-only file. Figure 3.6 shows the entity-relationship model, followed by a field-level description.

Entity: verdicts

Columns: id (PK, AUTOINCREMENT), created_at (TEXT, UTC ISO-8601), severity (TEXT), summary (TEXT), payload (TEXT, JSON of the full verdict and extras), signature (TEXT), fingerprint (TEXT). Each heartbeat appends one row; rows older than the retention window are pruned on insert.

Entity: acknowledgements

Columns: fingerprint (PK, TEXT), signature (TEXT), severity (TEXT), summary (TEXT), note (TEXT), acked_by (TEXT), created_at (TEXT), expires_at (TEXT, nullable). One row per acknowledged condition; an acknowledgement is active while expires_at is null or in the future.

Verdicts and acknowledgements are matched by fingerprint. Several verdict rows may share a fingerprint when the same condition recurs, while each fingerprint can have at most one active acknowledgement. A matching verdict is marked acknowledged. It is marked suppressed only when its severity is ok or warning.

3.4.2 Use-Case Model

Figure 3.4 shows the use-case model. Through the dashboard, the operator reviews current and historical verdicts, marks a low-risk condition as known, queries or re-indexes local runbooks, reviews detected threats, and may approve a CrowdSec IP ban. The heartbeat scheduler collects telemetry, obtains and stores a structured verdict from the configured model, and sends a high- or critical-severity alert when configured. Service remediation remains an operator action.

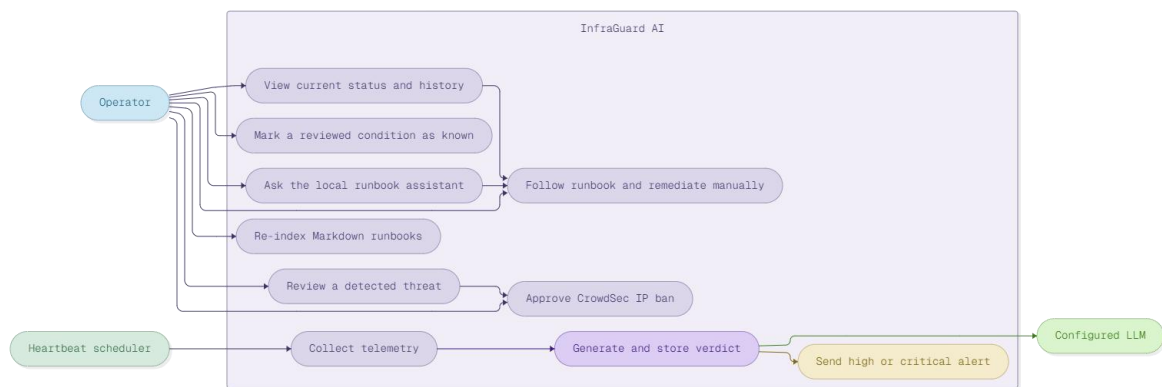


Figure 3.4: InfraGuard AI Use-Case Model.

3.4.3 Sequence Diagram: One Heartbeat Cycle

A heartbeat begins by loading active acknowledgements and collecting bounded telemetry from Loki, Prometheus, HTTP probes, and optional Docker sources, as shown in Figure 3.5. Known conditions are added to the model context. The returned verdict is validated, fingerprinted, and stored. When Docker monitoring is enabled, high or critical results may collect extra diagnostics; those severities can also trigger an ntfy alert. No service-changing command is executed by the heartbeat.

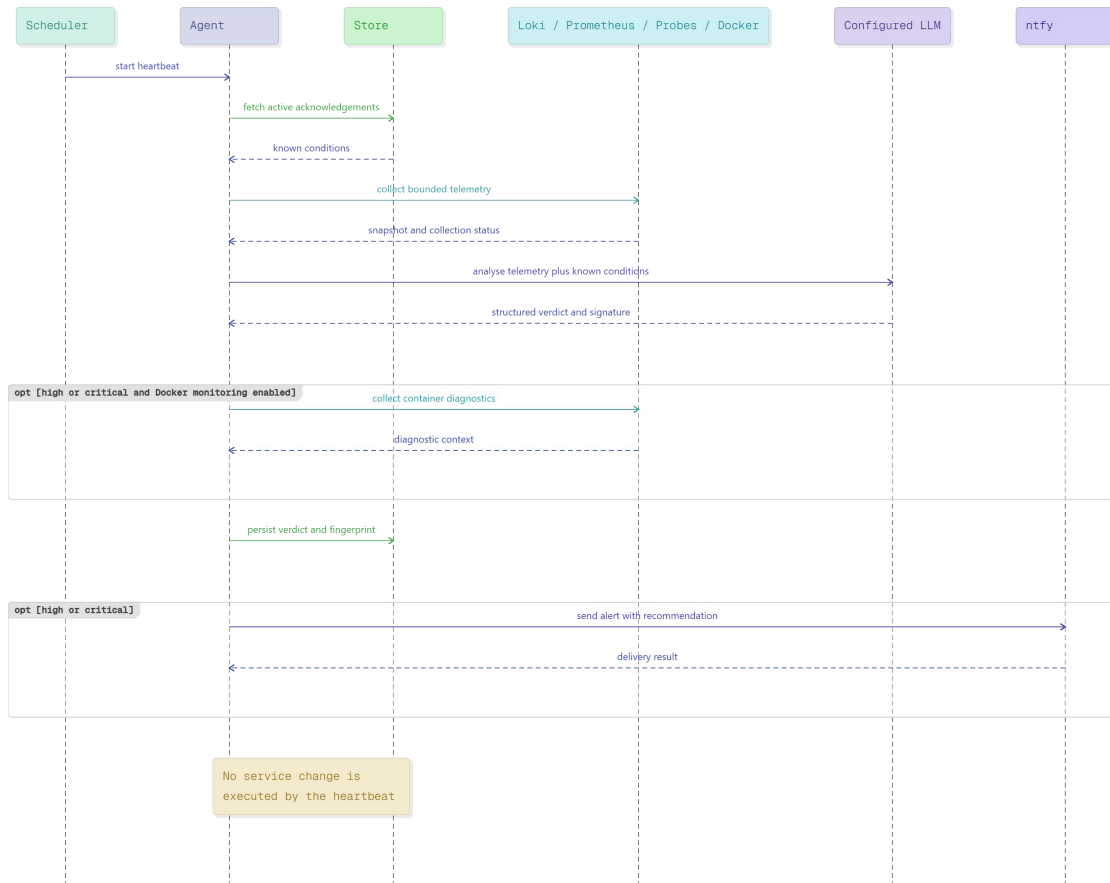


Figure 3.5: Sequence of One Heartbeat Cycle.

3.4.4 Entity-Relationship Model

Figure 3.6 visualises the two-table schema described above. Many verdict rows may carry the same fingerprint, while the acknowledgements table stores at most one row for that fingerprint together with its note, operator, creation time, and optional expiry.

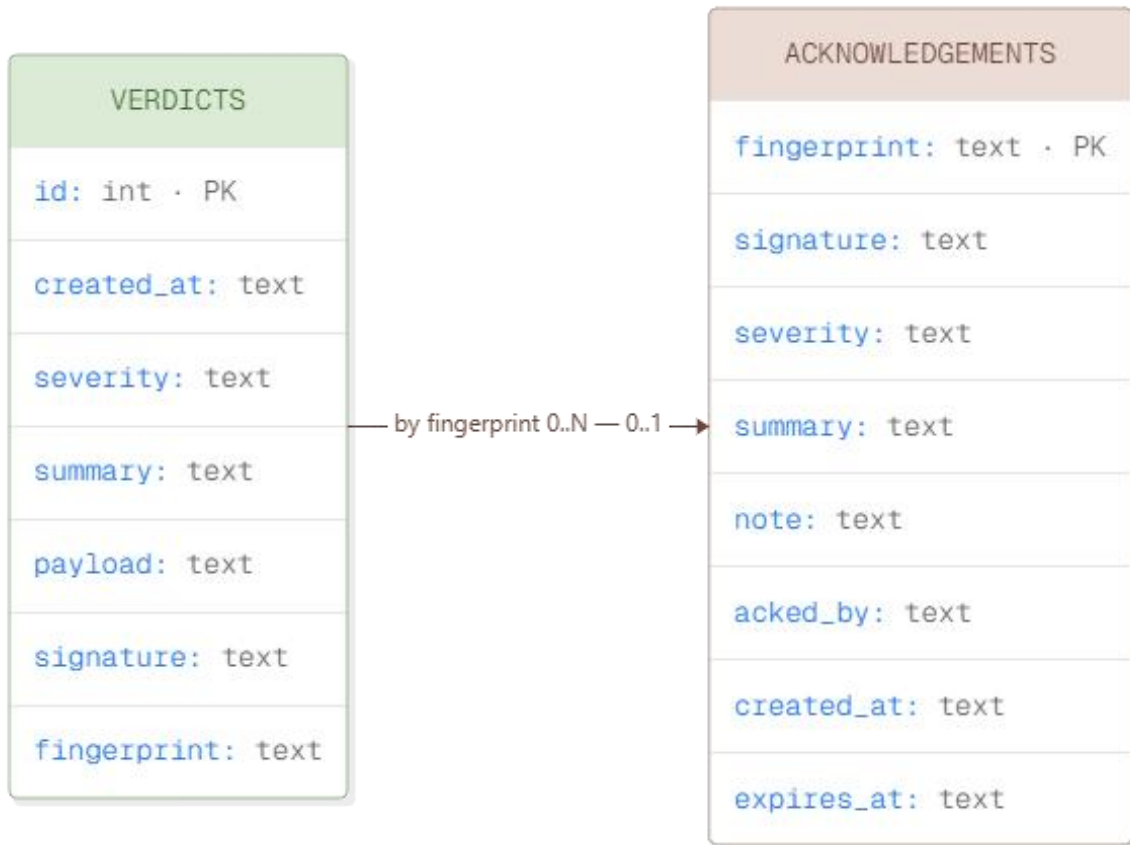


Figure 3.6: Verdict and Acknowledgement Data Model.

3.4.5 Verdict-Memory Design

The acknowledgement memory combines the condition signature, prompt version, and model in a SHA-256 fingerprint: $\text{fingerprint} = \text{SHA-256}(\text{signature} \mid \text{prompt_version} \mid \text{model})$. The model is asked to return a short signature such as "prometheus:disk-low:/". If it does not, the fallback removes IP addresses, hashes, and numbers from the root-cause or summary text before creating a slug. An operator acknowledgement stores the fingerprint with an optional note and expiry time. A changed signature, prompt version, or model produces a new fingerprint that does not match the previous acknowledgement. The previous database row remains until it expires or is removed. Matching ok and warning verdicts can be suppressed in the API and dashboard; high and critical verdicts cannot.

3.4.6 State Machine: LangGraph Reasoning Agent

Figure 3.7 depicts the LangGraph state machine that governs the reasoning agent. Four nodes, `collect_data`, `analyze`, `decide_action`, and `notify`, are used on each heartbeat. The graph begins at `collect_data`, proceeds to `analyze` and `decide_action`, and routes high- and critical-severity verdicts to `notify`. Other verdicts end without notification. When local Docker monitoring is

enabled, `decide_action` can also gather container diagnostics for high- and critical-severity verdicts.

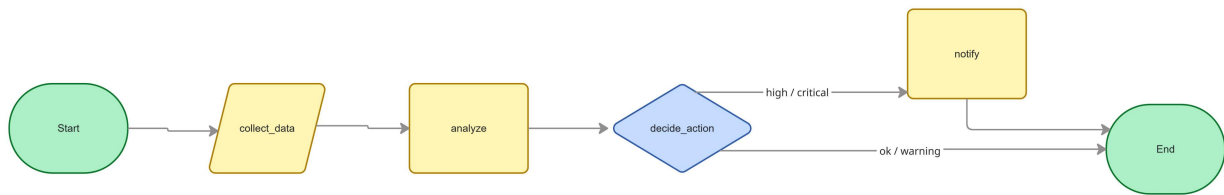


Figure 3.7: LangGraph Agent State Machine.

3.4.7 Workflow: Runbook RAG Indexing

Runbook indexing starts when the operator calls `POST /api/runbooks/index`. The loader recursively reads Markdown files from the configured local directory and retains title, source, and category metadata. It splits the documents into chunks, creates embeddings locally with the ONNX all-MiniLM-L6-v2 model, and stores the chunks in ChromaDB. For a query, the four closest chunks are supplied to the configured LLM with the operator's question, and the API returns a grounded answer with source titles. The operator reviews and executes any runbook steps manually.

3.5 Tools and Technologies Used

The implementation stack is grouped below by layer. Table 3.3 lists the telemetry sources and their collection modules.

3.5.1 Backend and Orchestration

- Language and runtime: Python 3.11+ with asyncio.
- Web/API framework: FastAPI with Uvicorn.
- LLM orchestration: LangGraph and LangChain.
- LLM providers: Gemini Developer API, Anthropic, OpenAI, or optional local Ollama, selected by configuration.
- Vector store and embeddings: ChromaDB with local ONNX all-MiniLM-L6-v2 embeddings.
- Persistence: SQLite via aiosqlite.
- HTTP client: httpx; runbook source: local Markdown files mounted read-only in the containers.

3.5.2 Security and Deployment

- Sessions: itsdangerous signed cookies; rate limiting: slowapi; security headers and CSP via custom middleware.
- Threat response: CrowdSec Local API.
- Notifications: ntfy.sh.
- Packaging and deployment: Docker Compose on a shared Linux VPS, Cloudflare Tunnel for HTTPS ingress, and GitHub Actions for CI.
- Frontend: a single self-contained HTML/CSS/JavaScript dashboard served by FastAPI.

3.5.3 Telemetry Sources

Table 3.3 enumerates the telemetry sources and the modules that collect them.

Source	Collection Module	Signal	Notes
Logs	agent/tools/loki.py	Recent log lines	Loki query_range, newest-first, configurable window
Metrics	agent/tools/prometheus.py	CPU, memory, disk, HTTP 5xx ratio	Prometheus instant queries
Endpoint health	agent/tools/http_probe.py	Status code and latency	Comma-separated PROBE_URLS
Container state	agent/tools/docker_api.py, docker_events.py	Events, stats, health, logs	Optional (ENABLE_DOCKER_MONITORING=1)
Threats	agent/tools/threat_response.py	Brute-force / port-scan patterns	Deterministic regex over recent logs
Local runbooks	agent/rag/local_runbooks_loader.py	Read-only Markdown corpus mounted into the containers.	Local Markdown source; no external documentation service

Table 3.3: Telemetry Sources and Collection Tools.

CHAPTER FOUR: SYSTEM IMPLEMENTATION

4.1 Development Environment

Development and validation used a workstation configured to approximate the intended single-host deployment.

4.1.1 Hardware and Operating System

Primary development was conducted on a Windows 11 workstation running Docker Desktop. All services run in Linux containers based on the `python:3.11-slim` image. The deployed environment is a single Linux VPS running native Docker, while the repository's Terraform definitions provide an alternative Google Compute Engine deployment path.

4.1.2 Source Control and Continuous Integration

The repository defines two GitHub Actions workflows. `test.yml` runs the pytest suite for pull requests to the main branch. `deploy.yml` repeats the tests after a push to main, then builds the agent and API images, publishes them to Google Artifact Registry, and updates the target virtual machine. Because these steps share one job, deployment proceeds only if the tests pass.

4.1.3 Editor and Tooling

Development was performed in Visual Studio Code with Python and Docker tooling. Application dependencies are listed in `requirements.txt` and installed into each container at build time.

4.2 Implementation Details

This section follows the repository layout; all paths are relative to the repository root.

4.2.1 Repository Layout

Figure 4.1 shows the folder structure, and Table 4.1 maps each path to its responsibility.

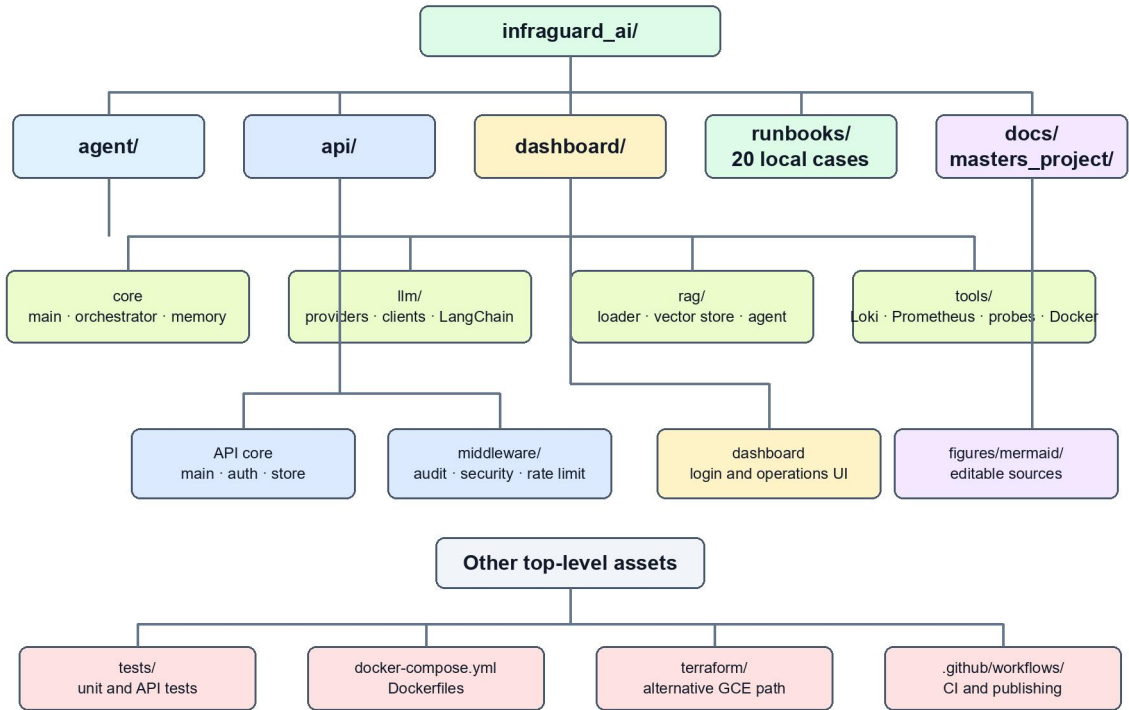


Figure 4.1: Repository Layout of the Reference Implementation.

Path	Responsibility
agent/main.py	Heartbeat entry point; fetches acknowledgements, runs the cycle, persists verdicts, dispatches notifications.
agent/orchestrator.py	The LangGraph state machine wiring collect_data, analyze, decide_action, and notify.
agent/memory.py	Verdict fingerprinting (signature + ruleset version) and deterministic signature fallback.
agent/llm/providers.py	Gemini, Anthropic, OpenAI-compatible and Ollama provider dispatch
agent/llm/schema.py	Shared verdict contract: system instruction, JSON extraction, and severity validation for every provider.
agent/llm/gemini_client.py	Gemini Developer API client
agent/llm/anthropic_client.py	Single-call Anthropic Claude client.
agent/llm/openai_client.py; agent/llm/ollama_client.py	OpenAI-compatible and self-hosted Ollama clients
agent/llm/langchain_agent.py	LangChain multi-tool agent (ReAct-style) over the telemetry tools.
agent/llm/prompts.py	Prompt assembly, evidence constraints, and the known-conditions memory block.
agent/rag/local_runbooks_loader.py	Loads Markdown runbooks and attaches category metadata
agent/rag/vector_store.py	ChromaDB binding: index, load, and similarity search.

Path	Responsibility
agent/rag/runbook_agent.py	Retrieval-augmented runbook question answering.
agent/tools/loki.py, prometheus.py, http_probe.py	Telemetry collection tools (logs, metrics, endpoint health).
agent/tools/docker_api.py, docker_events.py, docker_logs.py	Optional Docker Engine telemetry (events, diagnostics, log tail).
agent/tools/threat_response.py	Deterministic threat detection and the CrowdSec adapter.
agent/tools/notify.py, langchain_tools.py	ntfy.sh notifications; @tool wrappers for the multi-tool agent.
api/main.py	FastAPI application: routes, middleware registration, auth, and the acknowledgement and threat endpoints.
api/auth.py	Signed session-token creation and validation.
api/store.py	SQLite persistence for verdicts and acknowledgements, with retention pruning.
api/middleware/audit.py, security.py, rate_limit.py	Request auditing, security headers/CSP, and rate limiting.
dashboard/index.html, login.html	The single-page operations dashboard and the login page.
tests/	pytest suite: test_tools, test_agent, test_api, test_store, test_memory, test_rag, test_threat_response.
terraform/	Google Compute Engine infrastructure-as-code.
docker-compose.yml, agent/Dockerfile, api/Dockerfile	Container composition and image definitions.
.github/workflows/test.yml, deploy.yml	CI test workflow and CD deployment workflow.

Table 4.1: Repository Module Map and Responsibilities.

4.2.2 The Reasoning Agent and LangGraph Loop

The reasoning workflow lives in `agent/orchestrator.py` and is started on a schedule by the `asyncio` heartbeat in `agent/main.py`. At each interval, the heartbeat loads active acknowledgements and, when Docker monitoring is enabled, may tail error logs from a monitored container. The compiled LangGraph workflow runs on a worker thread so a blocking model call cannot stall the event loop. Its `collect_data` node gathers telemetry, analyze requests a verdict, and `decide_action` may add container diagnostics for a high or critical result. A conditional edge routes those two severities to notify; all other verdicts end the cycle.

4.2.3 The Two LLM Reasoning Modes

Two settings control LLM analysis: `USE_LANGCHAIN_AGENT` selects the analysis mode, and `LLM_PROVIDER` selects Gemini, Anthropic, OpenAI, or Ollama. The active model name is included in the acknowledgement fingerprint. Direct mode sends the assembled prompt in one request and parses the returned JSON through the shared schema module. In multi-tool mode, the provider chat model may call the Loki, Prometheus, and HTTP-probe tools before returning a verdict, subject to an iteration limit. Both paths produce the same normalised fields.

4.2.4 Prompt Construction and Response Validation

Prompt construction in `agent/llm/prompts.py` serialises only the telemetry sections that are available. It also instructs the model not to treat a missing optional integration as an incident. The shared parser in `agent/llm/schema.py` expects a JSON object and a recognised severity, then normalises the summary, root-cause, action, and signature fields. Missing text fields are replaced with empty strings and unexpected fields are ignored. This is a structured response contract with severity checking rather than a full JSON Schema validator.

4.2.5 The Verdict-Memory

Acknowledgement behaviour spans `agent/memory.py`, `api/store.py`, the prompt assembler, and the acknowledgement endpoints; Figure 4.2 traces the flow. The condition signature is combined with the prompt version and model name to produce a SHA-256 fingerprint. An operator may acknowledge an ok or warning verdict, add a note, and choose an expiry time. Active acknowledgements are passed back in later prompts as context, although every heartbeat still calls the model. When a later ok or warning verdict has the same fingerprint, the API marks it as suppressed and the dashboard hides it by default. A change in the signature, prompt version, or model creates a different fingerprint. Checks in the store and endpoints prevent high and critical verdicts from being suppressed.

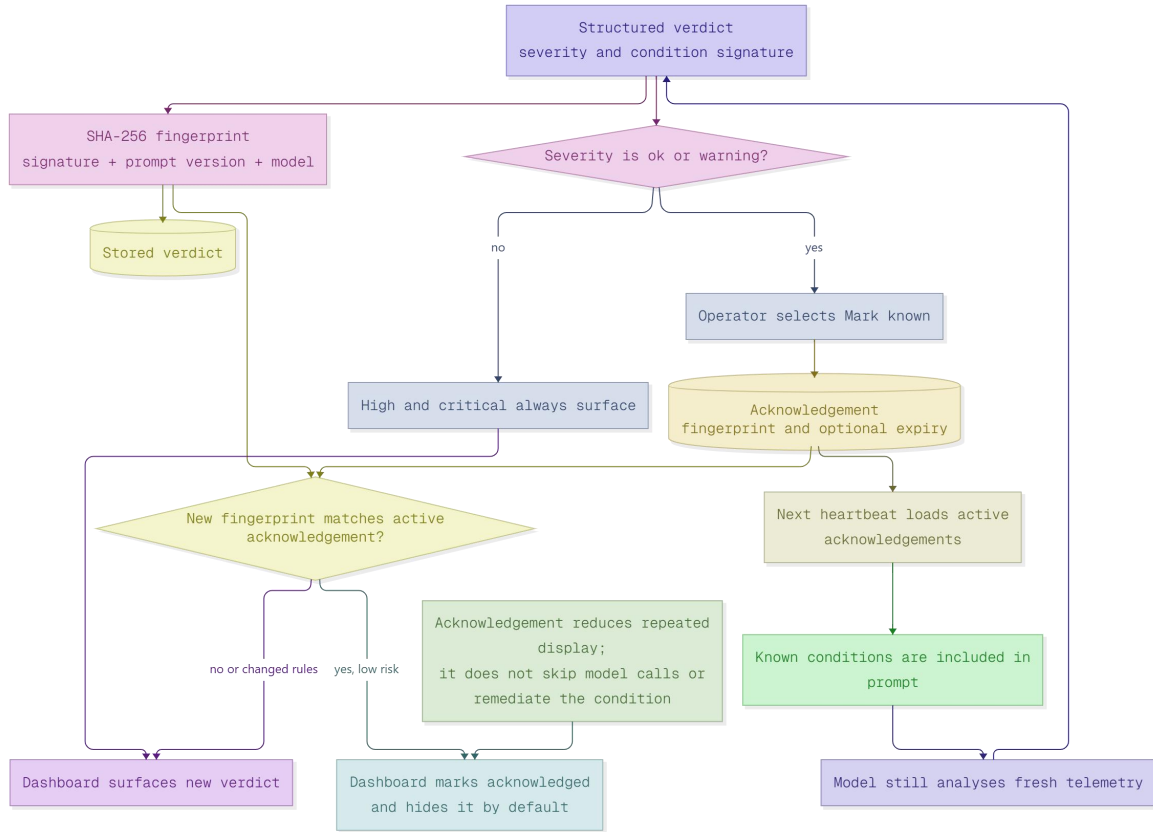


Figure 4.2: Verdict-Memory Fingerprint and Acknowledgement Flow.

4.2.6 Telemetry Collection Tools

Each telemetry adapter returns structured data or a structured error. The Loki adapter in `agent/tools/loki.py` uses `query_range` for a fixed recent window, orders results newest first, and enforces a configurable limit. Prometheus instant queries collect CPU, memory, disk, and HTTP 5xx-rate signals. HTTP probes record the status and latency of each configured endpoint. Docker events, diagnostics, and error logs are collected only when Docker monitoring is enabled.

4.2.7 The RAG Runbook Service

The runbook service in `agent/rag/` recursively loads Markdown files from `RUNBOOKS_DIR` while retaining title, source, and category metadata. It splits the text into chunks, creates embeddings locally with ONNX all-MiniLM-L6-v2, and stores the vectors in ChromaDB. At query time, the service retrieves the four closest chunks and supplies only that material to the configured LLM. The answer includes source titles. Operators trigger re-indexing from the dashboard, the runbooks are mounted read-only, and commands in the retrieved material are never executed by the application.

4.2.8 Threat Detection and CrowdSec Response

Threat detection in `agent/tools/threat_response.py` does not use an LLM. It counts repeated HTTP 401/403 responses, SSH authentication failures, and rapid connection attempts by source IP, then reports a threat when a configured threshold is crossed. If the operator approves a response, the API builds the CrowdSec ban decision from the recorded threat fields and submits it to the CrowdSec Local API. Without a configured CrowdSec endpoint, the same action is logged as a dry run.

4.2.9 The API Service and Middleware

The FastAPI application in `api/main.py` serves both the dashboard and the JSON REST API. Four middlewares are registered: audit logging, security headers, per-IP rate limiting, and authentication. The authentication layer requires a valid signed session cookie except on the public health and login endpoints, and the login route has its own stricter rate limit. Runbook queries, indexing, and Loki-backed threat scans run on worker threads so their blocking integration calls do not stall the event loop.

4.2.10 Persistence and Retention

SQLite persistence is implemented in `api/store.py` with `aiosqlite` and uses the verdicts and acknowledgements tables described in Section 3.4.1. The schema is created and migrated idempotently whenever a connection opens, allowing an existing database to gain the signature and fingerprint columns in place. Each insert also prunes verdicts older than the configured retention window, so no separate maintenance job is needed to bound storage growth.

4.2.11 The Dashboard

The dashboard (`dashboard/index.html`) is a single page served by FastAPI. It displays the current verdict, integration status, agent staleness, detected threats, runbook queries, and recent checks. Root-cause and recommended-action text can be expanded when needed.

For ok and warning verdicts, the page provides controls to mark or unmark a condition as known, a known tag, and a toggle for acknowledged rows. Text from the server, model, or logs passes through a single HTML-escaping helper before it is inserted into the page. This limits the cross-site-scripting risk posed by untrusted log or model content. Figure 4.3 shows the dashboard.

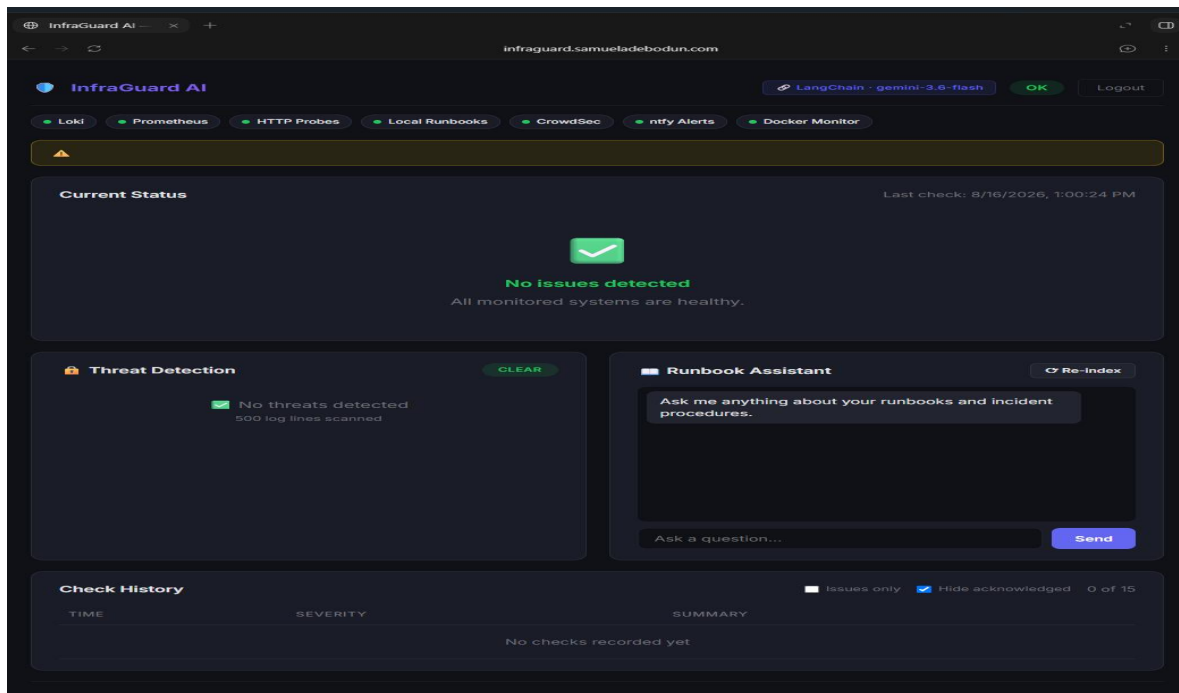


Figure 4.3: InfraGuard AI Operations Dashboard.

The dashboard runbook panel exposes the retrieval workflow described in Section 4.2.7. Figure 4.4 summarises local indexing, top-four retrieval, grounded answering, and the operator-review boundary.

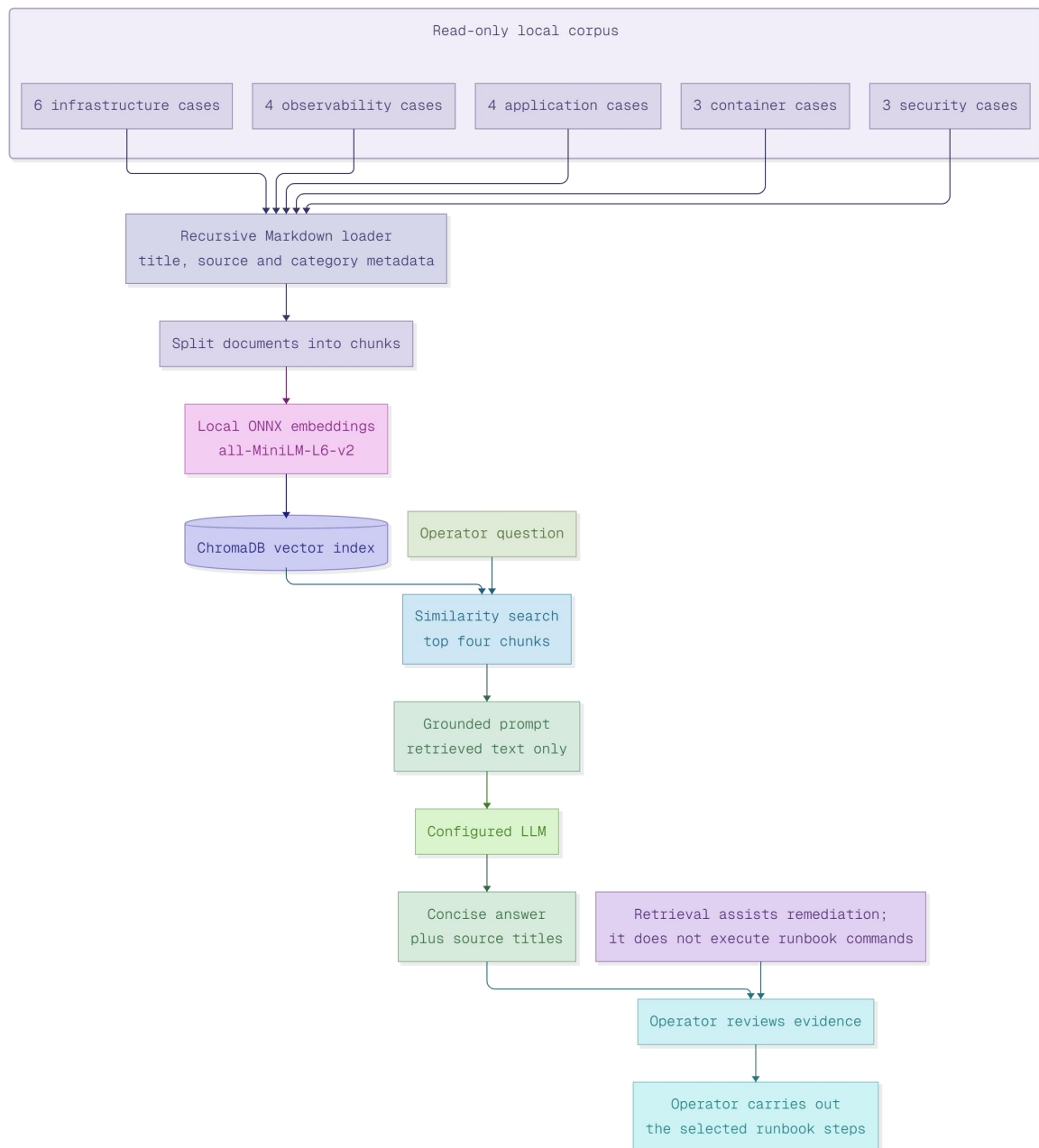


Figure 4.4: Local Runbook Indexing and Retrieval Process.

4.2.12 API Endpoints

Endpoint	Method	Purpose
/	GET	Serve the dashboard (authenticated).
/login, /logout	GET/POST	Session authentication; login is rate-limited to five attempts per minute.

Endpoint	Method	Purpose
/health	GET	Liveness check (public).
/status	GET	Latest verdict with staleness and memory (fingerprint, acknowledged, suppressed) fields.
/alerts	GET	Recent verdict history with per-row memory fields.
/api/config	GET	Which integrations are configured (booleans only).
/api/agent/mode	GET	Active reasoning mode and model.
/api/threats	GET	Scan recent Loki logs for brute-force and port-scan patterns.
/api/threats/apply	POST	Apply a CrowdSec ban for a detected threat (dry-run without CrowdSec).
/api/verdicts/ack	POST	Acknowledge a verdict condition as a known non-issue.
/api/verdicts/unack	POST	Remove an acknowledgement so the condition re-opens.
/api/acks	GET	List the active acknowledgements.
/api/runbooks/query	POST	Answer a question over the indexed runbooks (RAG).
/api/runbooks/index	POST	Clear the derived index and rebuild it from mounted local Markdown files.

Table 4.2: API Endpoint Reference.

4.3 Security, Performance, and Scalability Considerations

4.3.1 Authentication and Session Management

All non-public endpoints require a session cookie signed with an `itsdangerous` timestamped serialiser and an environment-provided secret. Sessions expire after twenty-four hours and the authentication middleware checks the cookie on every request. Credential comparison uses `hmac.compare_digest` to limit timing differences. The cookie is marked `HttpOnly` and is also marked `Secure` when the dashboard is served over HTTPS.

4.3.2 Rate Limiting and Output Hygiene

The `slowapi` middleware applies a general request limit and a stricter login limit. Dashboard rendering uses a shared escaping helper for server-derived text. The response middleware adds a content-security policy and other headers. The current policy permits inline script and style content because the dashboard is a single HTML file; this weakens the policy and should be removed in a hardened deployment by moving scripts and styles to separate files.

4.3.3 Auditability

The audit middleware writes one structured JSON line per request with timestamp, client IP, method, route, user, status, and elapsed time. The file is a local operational log. It is not

cryptographically protected or tamper-evident, so external log shipping and retention controls are recommended for production use.

4.3.4 Performance and Responsiveness

The agent and API run as separate processes. API routes that call blocking LLM, runbook-indexing, or Loki clients dispatch that work to a worker thread. The default heartbeat is 120 seconds, and one model analysis is performed per heartbeat. This design bounds request frequency, but latency and cost were not measured in the present evaluation.

4.3.5 Scalability and Degradation

The design targets one host and its co-located workloads; multi-host operation is future work. Each telemetry tool converts a failed request into a structured error, so the next prompt contains less evidence instead of terminating the cycle. A failed model call produces a warning fallback verdict so the failure is recorded.

4.3.6 Deployment Boundary

The supplied Terraform firewall currently exposes ports 80, 443, 3100, 9090, and 8080 to all source addresses, while Docker Compose publishes the API on port 8080 without a bundled TLS proxy. This is suitable only for a controlled demonstration network. A production deployment should expose only the HTTPS entry point, restrict administration and telemetry ports to trusted networks, set the secure-cookie option, and manage credentials outside the deployment directory.

4.4 Challenges Encountered

4.4.1 LLM Output Conformance

A provider response may contain markdown fences, surrounding prose, or an unsupported severity. The single-call mode requests application/json, and the shared parser extracts the outermost JSON object, normalises the expected fields, and checks the severity against the allowed values. A failed parse yields a structured error and a fallback verdict.

4.4.2 Scope Bounding and False Alarms

The prompt contains explicit scope rules for optional integrations because absent evidence can otherwise be interpreted as a failure. Automated tests confirm that prompt construction includes only configured sections and the required instruction. Live-model false-positive rates were not measured.

4.4.3 Signature Stability for the Memory

For the verdict memory to be useful, the condition signature must remain stable when volatile telemetry values change. The prompt requests a structured "<source>:<condition>:<resource>" signature at low temperature. If the model omits it, the

implementation derives a fallback after removing timestamps, IP addresses, numbers, and hashes.

4.4.4 Security Defects Corrected During Review

Implementation review identified two defects that were corrected: the rate limiter was configured without its middleware, and server-derived text was inserted into the dashboard without a shared escaping path. Regression tests cover the corrected behaviour. The remaining deployment and CSP limitations are recorded in Section 4.3.

4.4.5 Loki Query Semantics

An early implementation used Loki's instant-query endpoint and produced inconsistent recent-log results. The revised adapter uses a range query over an explicit window in newest-first order. This also gives the threat scanner enough log lines to evaluate its configured thresholds.

CHAPTER FIVE: TESTING, RESULTS AND EVALUATION

This chapter reports the verification evidence available at submission time: deterministic scenario checks, the automated test and coverage run recorded on 16 August 2026, and a limited live pilot of the deployed runbook assistant. The evidence establishes that the principal software paths and retrieval integration functioned in the tested environment. It does not support a claim of measured live-model diagnostic accuracy or practitioner usability.

5.1 Testing Strategy

Testing was organised at three levels. Unit tests cover individual functions, API integration tests exercise the FastAPI application, and focused tests check acknowledgement invariants. External HTTP services and provider clients are mocked so the suite can run repeatedly without credentials.

5.1.1 Unit Tests

Unit tests cover telemetry parsing, structured error returns, provider dispatch, verdict parsing, fingerprint generation, fallback-signature normalisation, deterministic threat detection, and selected runbook helpers. Mock responses represent Loki, Prometheus, HTTP endpoints, Docker, and provider clients.

5.1.2 API and Persistence Tests

FastAPI tests exercise login, authenticated routes, rate limiting, security headers, configuration output, threat routes, verdict history, acknowledgements, and staleness reporting. Persistence tests use temporary SQLite databases to verify inserts, retention pruning, acknowledgement expiry, suppression, and removal.

5.1.3 Acknowledgement Invariants

The memory tests check the full fingerprint rule. Identical signature, prompt-version, and model inputs must produce the same fingerprint, while changing any one of them must produce a different value. The tests also confirm that fallback signatures remove volatile tokens, active acknowledgements suppress only ok or warning verdicts, expired acknowledgements are inactive, and high or critical verdicts remain visible.

5.2 Automated Test Results

The complete automated suite passed 82 tests with 17 warnings in 27.01 seconds. Coverage.py reported 1,476 executable statements, 508 missed statements and 66% overall line coverage. Coverage was strongest in acknowledgement memory (100%), orchestration (95%), provider dispatch (93%) and persistence (94%). Lower results remained in the

LangChain agent path (15%), vector store (31%), Docker API path (12%), Docker events (14%) and the agent entry point (0%).

5.2.1 Coverage by Module

Module	Line Coverage	Interpretation
agent/memory.py	100%	Acknowledgement fingerprint and memory
agent/orchestrator.py	95%	Collect-analyse-decide-notify workflow
api/store.py	94%	SQLite verdict and acknowledgement storage
agent/llm/schema.py	95%	Structured verdict validation
agent/llm/providers.py	93%	Provider selection and dispatch
api/middleware/security.py	100%	Security-header middleware
api/middleware/rate_limit.py	100%	API rate limiting
api/middleware/audit.py	97%	Audit-event middleware
api/main.py	71%	Dashboard and API routes
api/auth.py	82%	Login and session handling
agent/tools/loki.py	85%	Loki query client
agent/tools/prometheus.py	80%	Prometheus query client
agent/tools/docker_logs.py	81%	Container log collection
agent/tools/http_probe.py	70%	HTTP availability probes
agent/tools/threat_response.py	81%	Threat scanning and response logic
agent/llm/langchain_agent.py	15%	LangChain multi-tool path
agent/rag/vector_store.py	31%	ChromaDB indexing and retrieval
agent/rag/local_runbooks_loader.py	89%	Local Markdown loader
agent/llm/gemini_client.py	88%	Gemini Developer API client
agent/main.py	0%	Scheduled agent entry point
Overall: 1,476 statements; 508 missed; 82 tests	66%	Measured 16 August 2026; 82 tests passed

Table 5.1: Selected Module Coverage from the Verification Run (measured 16 August 2026).

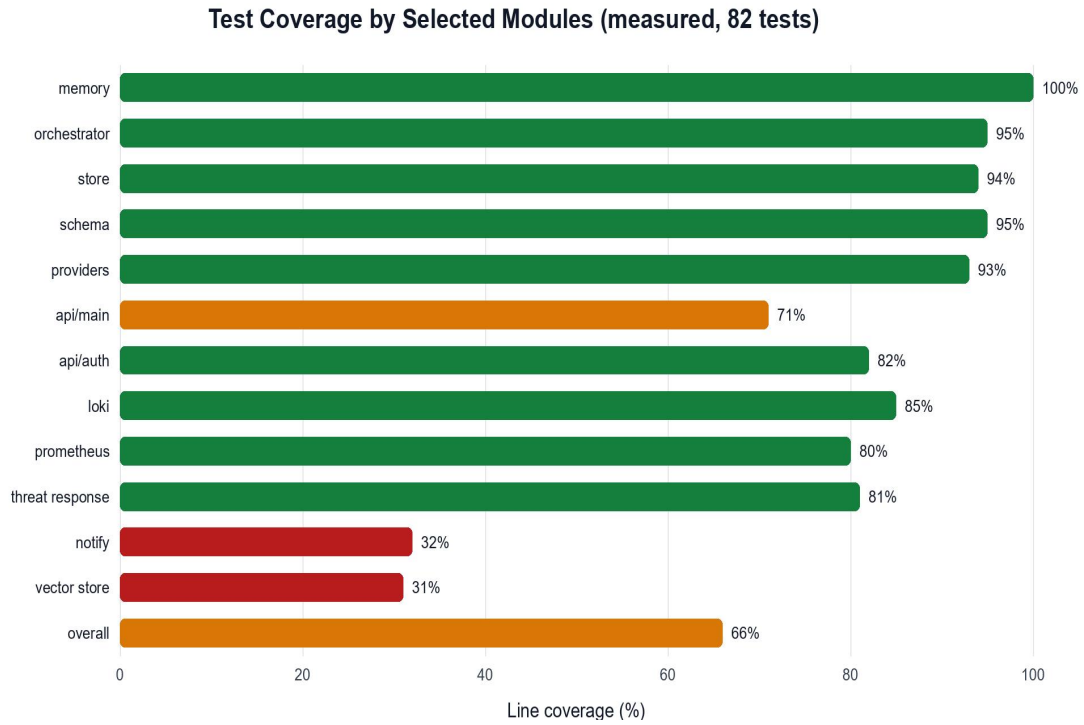


Figure 5.1: Selected Module Coverage from the Verification Run.

5.2.2 Representative Test Cases

ID	Scenario	Expected Outcome	Result
TC-01	Loki range query returns multiple streams	Recent lines are parsed in newest-first order	Pass
TC-02	Prometheus collection	Configured metric keys are returned	Pass
TC-03	HTTP probe	Status and latency are recorded for each URL	Pass
TC-04	Critical verdict routing	Notification path is selected	Pass
TC-05	OK verdict routing	Notification path is skipped	Pass
TC-06	Fingerprint stability	Identical inputs yield the same fingerprint	Pass
TC-07	Fingerprint mismatch	Signature, prompt, or model change yields a different fingerprint	Pass
TC-08	Severity safeguard	High and critical verdicts are not suppressed	Pass
TC-09	Warning acknowledgement	Matching warning is marked acknowledged and suppressed	Pass
TC-10	Expired acknowledgement	Expired row is not active	Pass
TC-11	Critical acknowledgement request	API returns HTTP 409	Pass
TC-12	Retention pruning	Rows outside the window are removed on insert	Pass

ID	Scenario	Expected Outcome	Result
TC-13	HTTP brute-force fixture	Threshold crossing produces a threat record	Pass
TC-14	Login rate limit	Sixth rapid failed attempt returns HTTP 429	Pass
TC-15	Stale verdict	Old verdict is reported as stale	Pass
TC-16	Configuration response	Only integration booleans are returned	Pass

Table 5.2: Representative Automated Test Cases and Results.

5.3 Acknowledgement-Memory Evaluation

Acknowledgement changes how repeated low-risk verdicts are presented. In the 30-cycle scenario, the same warning fingerprint is stored on every cycle. Without an acknowledgement, all 30 rows remain visible. If the operator acknowledges the condition after cycle 3, the next 27 model calls and verdict records still occur, but their matching warnings are hidden from the default view. The visible cumulative count therefore remains at 3. Figure 5.2 plots both cases, while TC-08 and TC-09 test the rules behind the scenario.

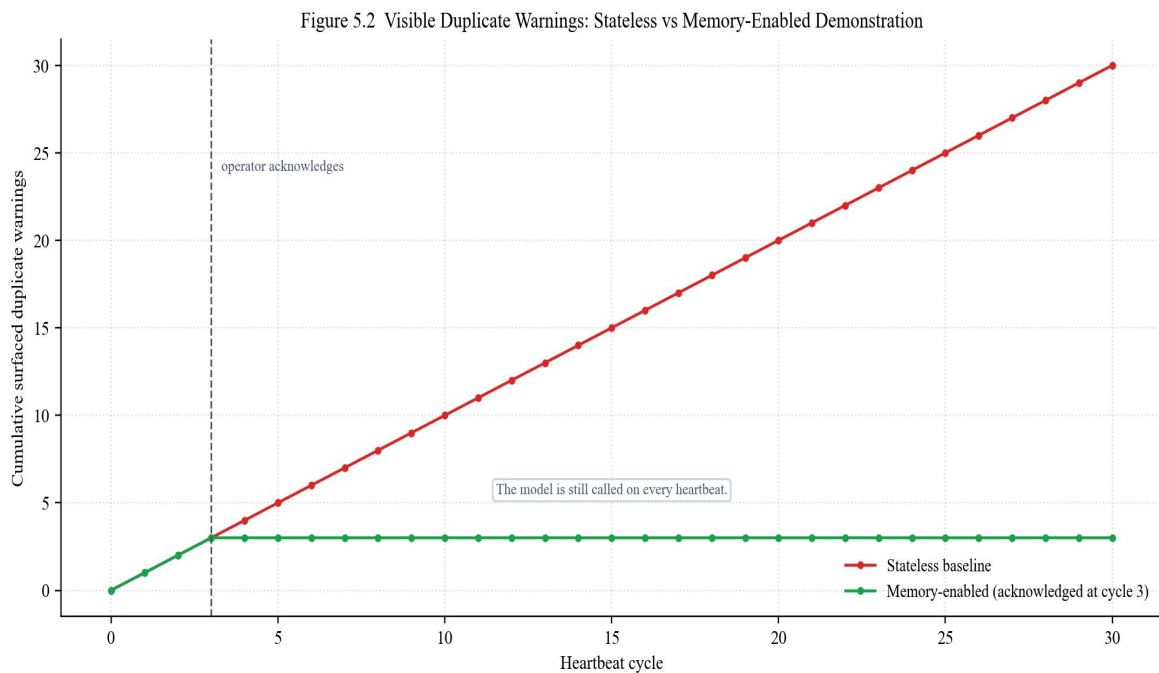


Figure 5.2: Visible Duplicate Warnings With and Without Acknowledgement.

This result supports a narrow conclusion: the feature reduces repeated operator-facing warnings in the default dashboard view. It does not reduce the number of model calls, the number of stored heartbeat results, or inference cost.

5.4 Limited Live Functional Verification

5.4.1 Runbook Retrieval Pilot

On 16 August 2026, the deployed dashboard re-indexed twenty local Markdown runbooks and used DeepSeek V4 Flash through the OpenAI-compatible LangChain path for three illustrative operator questions concerning disk capacity (RB-01), memory pressure (RB-03) and Loki ingestion (RB-08). In each observed interaction, the assistant returned actionable guidance aligned with the expected runbook subject. This confirms that local loading, ONNX embedding, ChromaDB retrieval, provider invocation and dashboard rendering worked together in the deployed configuration.

This was a convenience pilot rather than a controlled accuracy study. The three questions were not independently scored; response latency, token use and cost were not recorded; and no no-RAG baseline was executed. The heartbeat agent was intentionally stopped during the question-answering session, so the stale-status banner remained visible. The pilot therefore does not establish live telemetry diagnostic accuracy. Figure 5.3 retains the five-case workflow required for a later controlled evaluation.

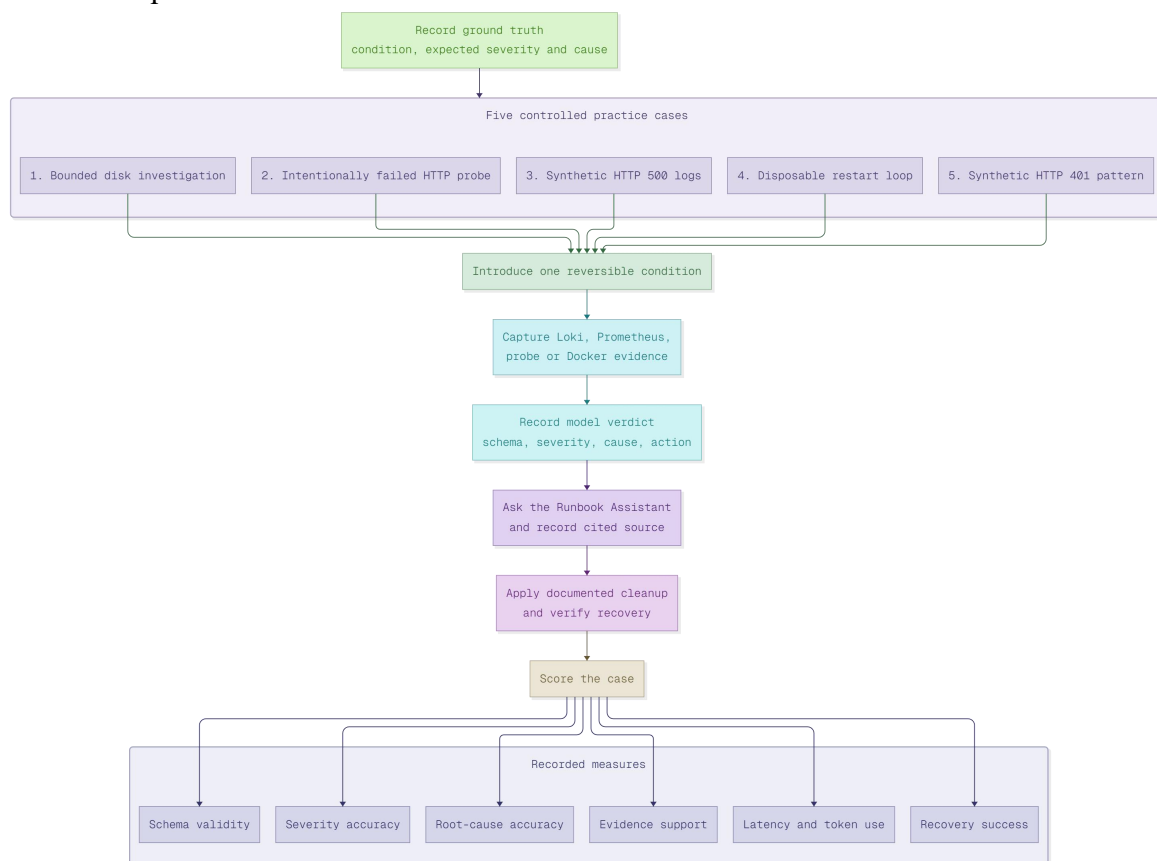


Figure 5.3: Five-Case Practice Evaluation Workflow.

5.4.2 Practitioner Usability Evaluation Not Completed

No structured practitioner study was completed. The System Usability Scale instrument in Appendix D remains a proposed protocol and no usability score is reported.

5.5 Evaluation Against the Objectives

Obj.	Objective	Evidence and Qualification	Status
O-1	Multi-source telemetry collection.	Loki, Prometheus, HTTP, and optional Docker adapters are implemented and tested with mocked upstreams.	Achieved
O-2	Structured LLM reasoning workflow.	The graph, provider dispatch, parser, and routing are implemented and tested. Live diagnostic accuracy was not evaluated.	Partly achieved
O-3	Runbook retrieval and operator-approved threat response.	The deployed pilot indexed 20 local runbooks and returned subject-aligned guidance for three illustrative questions. A controlled retrieval benchmark and live CrowdSec action test were not completed.	Partly achieved
O-4	Fingerprint-based acknowledgement memory.	Fingerprint, expiry, API suppression, removal, and severity safeguards are covered by tests and the deterministic comparison.	Achieved
O-5	Single-host packaging and evaluation.	82 automated tests passed with 66% line coverage. A limited deployed runbook pilot was completed; diagnostic accuracy, cost, latency and usability evaluation remain outstanding.	Partly achieved

Table 5.3: Objective and Evidence Matrix.

5.6 Discussion and Threats to Validity

5.6.1 Supported Findings

The supported findings are therefore limited but useful. Deterministic tests verify the condition fingerprint and acknowledgement behaviour; the automated suite verifies the main API, storage, orchestration, telemetry and provider-dispatch paths; and the deployed pilot verifies that twenty local runbooks can be indexed and used in live model-assisted retrieval. Together, these results demonstrate implementation feasibility, not comparative model quality or production effectiveness.

5.6.2 Limitations

The principal limitation is the absence of a controlled live telemetry diagnostic benchmark. Overall line coverage is 66%, but important execution paths remain lightly exercised: agent/main.py recorded 0%, the LangChain agent 15%, the vector store 31%, Docker API 12% and Docker events 14%. External cloud services and operational failure conditions are still mocked in much of the automated suite. The live retrieval pilot contained only three convenience questions, without independent scoring, a no-RAG baseline, repeated trials, latency, token or cost measurements. No practitioner usability study was conducted.

5.6.3 Validity

Construct validity is strongest for software behaviour directly represented by assertions, such as fingerprint stability, API responses and persistence. It is weaker for diagnostic quality because passing tests do not establish that a model has identified the correct operational root cause. The three-question retrieval pilot also has limited internal and external validity: the sample was small, selected by the developer, not independently scored and not compared with an ungrounded model. The conclusions are consequently restricted to verified implementation and limited live integration.

CHAPTER SIX: SUMMARY, CONCLUSION AND RECOMMENDATIONS

6.1 Summary of the Project

The project began with a practical gap between collecting operational telemetry and interpreting it. InfraGuard AI addresses that gap as a single-host reference system built from a scheduled agent, FastAPI service, browser dashboard, SQLite store, and optional runbook and threat-response integrations. The agent gathers the configured Loki, Prometheus, HTTP, and Docker evidence, then asks the selected LLM provider for a structured verdict through a LangGraph workflow.

The final artefact supports Gemini Developer API, Anthropic, OpenAI-compatible endpoints and Ollama through one provider-dispatch interface. The deployed retrieval pilot used DeepSeek V4 Flash through the OpenAI-compatible route, demonstrating that a compatible third-party model could be selected without changing the runbook pipeline.

The acknowledgement feature computes a fingerprint from the condition signature, prompt version, and model. An active match can mark an ok or warning verdict as suppressed in the API and dashboard. A changed input produces a different fingerprint, and high or critical verdicts are never suppressed. The workflow continues to call the LLM and store a verdict on every heartbeat, so the feature reduces repeated display rather than inference.

The recorded verification run produced 82 passing tests and 66% overall line coverage. A limited deployed pilot indexed twenty runbooks and produced relevant responses for three illustrative incident questions. This evidence supports claims about implementation and integration only; diagnostic accuracy, comparative provider performance, cost, latency and practitioner usability were not measured.

6.2 Conclusion

The completed system connects a single-host, LLM-assisted triage workflow to Loki, Prometheus, HTTP probes, and optional Docker telemetry. It records structured verdicts while keeping acknowledgement and threat-response actions under the operator's control.

The acknowledgement design provides a practical way to label repeated low-risk conditions without hiding high-risk results. Its safety depends on the quality and stability of the condition signature. The severity safeguard and expiry rules reduce risk, but they do not remove the need for operator review.

The evidence supports describing InfraGuard AI as a tested reference implementation. It does not support claims that prompt constraints prevent hallucinations, that the system lowers model cost, or that operators find the dashboard usable. Each of those questions needs a separate live-model or participant study.

6.3 Professional Contributions

6.3.1 Reproducible Reference Implementation

The repository shows how telemetry adapters, a provider-independent LLM interface, LangGraph orchestration, persistence, an authenticated API, a dashboard, and automated tests can be assembled for a single-host operations use case.

6.3.2 Acknowledgement Pattern

The content-and-ruleset fingerprint links an acknowledgement to the observed condition and the active reasoning configuration. A similar design may suit other systems that must separate reviewed low-risk results from new findings, provided they also enforce an independent severity safeguard.

6.3.3 Documented Limits

The project does not claim autonomous remediation or proven production reliability. Full heartbeat-based diagnostic evaluation, a labelled incident benchmark, comparative provider testing, a larger runbook-retrieval study, cost and latency profiling, practitioner usability assessment and production security hardening remain incomplete.

6.4 Recommendations for Future Work

6.4.1 Evaluation

1. Create a labelled set of healthy, degraded, and failing telemetry scenarios. Measure severity accuracy, root-cause accuracy, evidence support, false positives, false negatives, schema-valid output rate, latency, and cost for each provider and model version.

2. Administer the Appendix D tasks and System Usability Scale to an approved participant group. Report participant characteristics, task outcomes, times, errors, SUS calculations, interview themes, and interface changes.
3. Run a longer field evaluation to measure operator-facing duplicate reduction, acknowledgement errors, and the effect of signature changes. Do not use stored-verdict suppression as a proxy for reduced inference cost.

6.4.2 Test and Security Coverage

1. Future tests should exercise the local runbook loader, embedding and ChromaDB retrieval path with a labelled question set, independently scored relevance criteria and a no-RAG baseline. The same work should measure end-to-end latency, token use and cost for hosted models, together with capacity and response-time measurements for Ollama.
2. Place the API behind an HTTPS reverse proxy, publish only the HTTPS port, restrict Prometheus, Loki, and administration ports to trusted networks, set secure cookies, and move credentials to a managed secret store.
3. Replace inline dashboard scripts and styles so the content-security policy no longer requires unsafe-inline. Send audit events to a protected external store if tamper evidence is required.

6.4.3 Architecture

1. Evaluate deterministic condition signatures derived before the LLM call. If reliable, they could allow some acknowledged conditions to bypass model analysis, after appropriate severity and freshness checks.
2. Evaluate and harden the existing self-hosted Ollama path where data-control requirements prohibit cloud inference. The evaluation should compare model quality, response time and resource cost with the hosted providers.
3. Treat autonomous remediation and multi-host operation as separate projects with their own threat models, approval policies, rollback mechanisms, and evaluations.

REFERENCES

- Ahmed, T., Ghosh, S., Bansal, C., Zimmermann, T., Zhang, X., & Rajmohan, S. (2023). Recommending root-cause and mitigation steps for cloud incidents using large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (pp. 1737-1749). IEEE.
<https://doi.org/10.1109/ICSE48619.2023.00149>
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.
- Brooke, J. (1996). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189-194). Taylor & Francis.
- Central Bank of Nigeria. (2024). *Risk-based cybersecurity framework and guidelines for deposit money banks and payment service banks*.
https://www.cbn.gov.ng/Out/2024/BSD/CBN%20Risk-Based%20Cybersecurity%20Framework%20for%20DMBs%20and%20PSBs_2024.pdf
- Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., & Xu, T. (2024). Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems* (pp. 674-688). Association for Computing Machinery.
<https://doi.org/10.1145/3627703.3629553>
- Chen, Y., Shetty, M., Somashekar, G., Ma, M., Simmhan, Y., Mace, J., Bansal, C., Wang, R., & Rajmohan, S. (2025). AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds. *Proceedings of Machine Learning and Systems*, 7.
https://proceedings.mlsys.org/paper_files/paper/2025/hash/d1f9e4a9f109b6e8b75ed362736f22ec-Abstract-Conference.html
- Datadog. (n.d.). *Knowledge sources for Bits Investigation*. Retrieved July 30, 2026, from https://docs.datadoghq.com/bits_ai/bits_investigation/knowledge_sources/
- Federal Republic of Nigeria. (2023). *Nigeria Data Protection Act, 2023*. Nigeria Data Protection Commission. <https://ndpc.gov.ng/download/nigeria-data-protection-act-2023>
- Gemini 2.5 Team. (2025). *Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities*. [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2507.06261>

- Grafana Labs. (n.d.). *Grafana Loki documentation*. Retrieved July 30, 2026, from <https://grafana.com/docs/loki/latest/>
- Guo, H., Yang, J., Liu, J., Yang, L., Chai, L., Bai, J., Peng, J., Hu, X., Chen, C., Zhang, D., Shi, X., Zheng, T., Zheng, L., Zhang, B., Xu, K., & Li, Z. (2024). OWL: A large language model for IT operations. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=SZOQ9RKYJu>
- He, P., Zhu, J., Zheng, Z., & Lyu, M. R. (2017). Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE International Conference on Web Services (ICWS)* (pp. 33-40). IEEE. <https://doi.org/10.1109/ICWS.2017.13>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75-105. <https://doi.org/10.2307/25148625>
- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), Article 220. <https://doi.org/10.1145/3695988>
- International Organization for Standardization. (2022). *ISO/IEC 27001:2022 information security, cybersecurity and privacy protection: Information security management systems: Requirements*. <https://www.iso.org/standard/27001>
- Jin, P., Zhang, S., Ma, M., Li, H., Kang, Y., Li, L., Liu, Y., Qiao, B., Zhang, C., Zhao, P., He, S., Sarro, F., Dang, Y., Rajmohan, S., Lin, Q., & Zhang, D. (2023). Assess and summarize: Improve outage understanding with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (pp. 1657-1668). Association for Computing Machinery. <https://doi.org/10.1145/3611643.3613891>
- k8sgpt. (n.d.). *Getting started guide*. Retrieved July 30, 2026, from <https://docs.k8sgpt.ai/getting-started/getting-started/>
- LangChain. (n.d.). *LangGraph overview*. Retrieved July 30, 2026, from <https://docs.langchain.com/oss/python/langgraph/overview>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474. <https://arxiv.org/abs/2005.11401>
- National Institute of Standards and Technology. (2024). *Artificial intelligence risk management framework: Generative artificial intelligence profile (NIST AI 600-1)*. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.AI.600-1>

- New Relic. (n.d.). *Meet New Relic AI, your observability assistant*. Retrieved July 30, 2026, from <https://docs.newrelic.com/docs/agent-ai/new-relic-ai/>
- OpenAPI Initiative. (2025). *OpenAPI specification, version 3.1.2*.
<https://spec.openapis.org/oas/v3.1.2.html>
- OWASP Foundation. (2025). *OWASP Top 10:2025*. <https://owasp.org/Top10/>
- PagerDuty. (n.d.). *PagerDuty AIOps*. Retrieved July 30, 2026, from
<https://support.pagerduty.com/main/docs/aiops>
- Prometheus Authors. (n.d.). *Exposition formats*. Retrieved July 30, 2026, from
https://prometheus.io/docs/instrumenting/exposition_formats/
- Roy, D., Zhang, X., Bhawe, R., Bansal, C., Las-Casas, P. H. B., Fonseca, R., & Rajmohan, S. (2024). Exploring LLM-based agents for root cause analysis. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (pp. 208-219). Association for Computing Machinery.
<https://doi.org/10.1145/3663529.3663841>
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 8634-8652. <https://doi.org/10.52202/075280-0377>
- Xu, J., Zhang, Q., Zhong, Z., He, S., Zhang, C., Lin, Q., Pei, D., He, P., Zhang, D., & Zhang, Q. (2025). OpenRCA: Can large language models locate the root cause of software failures? In *International Conference on Learning Representations*.
https://proceedings.iclr.cc/paper_files/paper/2025/hash/d29b8d53678015079e1d245c023e49d2-Abstract-Conference.html
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.
https://openreview.net/forum?id=WE_vluYUL-X
- Yuan, P., Tang, L., Liu, Y., Kobayashi, Y., Sato, M., & Chen, H. (2025). Incident diagnosing and reporting system based on retrieval augmented large language model. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(28), 29721-29723.
<https://doi.org/10.1609/aaai.v39i28.35379>

APPENDIX A: API ENDPOINT REFERENCE

Appendix A lists the HTTP endpoints exposed by the InfraGuard AI API. A dagger marks routes that require a valid session cookie. Only the health check and login routes are public.

A.1 Authentication and Health

GET /login: Serve the login page. POST /login: Submit credentials; sets a signed session cookie on success; rate-limited to five attempts per minute.

GET /logout †: Clear the session cookie.

GET /health: Liveness probe; returns 200 when the API is reachable.

A.2 Status, History, and Configuration

GET /status †: The latest verdict with staleness (age, stale) and memory (fingerprint, acknowledged, suppressed) fields.

GET /alerts †: The twenty most recent verdicts with their per-row memory fields.

Local runbooks: Markdown files mounted read-only from ./runbooks and indexed from RUNBOOKS_DIR=/app/runbooks.

GET /api/agent/mode †: The active reasoning mode (LangChain or direct) together with the selected provider and model.

A.3 Verdict Memory

POST /api/verdicts/ack †: Acknowledge a verdict condition as a known non-issue, keyed on its fingerprint; rejected with HTTP 409 for high/critical severity.

POST /api/verdicts/unack †: Remove an acknowledgement so the condition re-opens.

GET /api/acks †: List the active acknowledgements.

A.4 Threats

GET /api/threats †: Scan recent Loki logs for HTTP/SSH brute-force and port-scan patterns; returns detected threats and source IPs.

POST /api/threats/apply †: Build and apply a CrowdSec ban for a detected threat; runs in dry-run mode when CrowdSec is not configured.

A.5 Runbooks (RAG)

POST /api/runbooks/query †: Answer a question over the indexed runbooks; returns an answer and source citations.

Use the Re-index control to clear the derived ChromaDB index and rebuild it from the Markdown files currently mounted under /app/runbooks.

APPENDIX B: SAMPLE CODE SNIPPETS

The excerpts below show the fingerprint, severity safeguard, known-condition prompt block, threat detector, and LangGraph workflow. The project repository contains the complete source.

B.1 Verdict Fingerprint and Signature Derivation

From agent/memory.py, the condition-and-ruleset fingerprint used by the acknowledgement feature:

```
import hashlib, re

PROMPT_VERSION = "1"          # bump to invalidate all prior acknowledgements
DEFAULT_MODEL = "gemini-3.6-flash"
OK_SIGNATURE = "none:healthy:all"

def compute_fingerprint(signature, model=DEFAULT_MODEL,
                        prompt_version=PROMPT_VERSION):
    """Stable content-plus-ruleset fingerprint for a verdict condition."""
    sig = (signature or "").strip().lower() or OK_SIGNATURE
    basis = f"{sig}\x1f{prompt_version}\x1f{model}"
    return hashlib.sha256(basis.encode("utf-8")).hexdigest()

def derive_signature_fallback(severity, root_cause="", summary=""):
    """Deterministic signature when the model does not supply one;
    volatile tokens (IPs, numbers, hashes) are stripped so the same
    condition normalises to the same string across heartbeats."""
    sev = (severity or "warning").strip().lower()
    if sev == "ok":
        return OK_SIGNATURE
    basis = _normalize(root_cause) or _normalize(summary)
    slug = _NONWORD.sub("-", basis).strip("-")[:48] or "unspecified"
    return f"{sev}:{slug}"
```

B.2 Severity Safeguard

From `api/store.py`, the rule that prevents a high or critical verdict from being suppressed:

```
def is_suppressed(severity, acknowledged):
    """An acknowledged condition is hidden only when low-risk;
    high/critical always surface, acknowledged or not."""
    return acknowledged and str(severity or "").lower() in {"ok", "warning"}
```

B.3 Known-Conditions Memory Block

From `agent/llm/prompts.py`, active acknowledgements included in the next prompt as contextual information:

```
def _known_conditions_block(known_conditions):
    if not known_conditions:
        return ""
    lines = []
    for kc in known_conditions:
        sig = str(kc.get("signature") or "").strip()
        if not sig:
            continue
        note = str(kc.get("note") or "").strip()
        lines.append(f"- {sig}" + (f"; operator note: {note}" if note else ""))
    if not lines:
        return ""
    return (
        "OPERATOR-ACKNOWLEDGED KNOWN CONDITIONS (already triaged and "
        "accepted as non-issues; do NOT re-flag or escalate these unless "
        "the underlying condition has materially changed). If a current "
        "observation matches one of these and has not materially changed, "
        "return severity \"ok\" and reuse its signature:\n"
        + "\n".join(lines) + "\n\n"
    )
```

B.4 Deterministic Threat Detection

From `agent/tools/threat_response.py`, the deterministic per-source-IP threshold detector:

```
def analyze_threats(loki_logs):
    ip_401_count, ip_ssh_fail = {}, {}
    for entry in loki_logs:
        line = str(entry.get("line", ""))
        low = line.lower()
        if " 401 " in line or " 403 " in line or "unauthorized" in low:
            for ip in _extract_ips(line):
                ip_401_count[ip] = ip_401_count.get(ip, 0) + 1
        if "failed password" in low or "authentication failure" in low:
            for ip in _extract_ips(line):
```

```

        ip_ssh_fail[ip] = ip_ssh_fail.get(ip, 0) + 1
threats = []
for ip, n in ip_401_count.items():
    if n >= _BRUTE_FORCE_THRESHOLD:
        threats.append({"threat_type": "http_brute_force",
                        "source_ip": ip, "count": n})
# ... ssh and port-scan thresholds elided ...
return {"threats_found": bool(threats), "threats": threats}

```

B.5 LangGraph Reasoning Loop

From agent/orchestrator.py, the four-node state machine:

```

def build_graph():
    graph = StateGraph(GraphState)
    graph.add_node("collect_data", _collect_data)
    graph.add_node("analyze", _analyze)
    graph.add_node("decide_action", _decide_action)
    graph.add_node("notify", _notify)
    graph.add_edge(START, "collect_data")
    graph.add_edge("collect_data", "analyze")
    graph.add_edge("analyze", "decide_action")
    graph.add_conditional_edges("decide_action", _route_notify,
                               {"notify": "notify", "skip": END})
    graph.add_edge("notify", END)
    return graph.compile()

```

APPENDIX C: DOCKER COMPOSE CONFIGURATION

The docker-compose.yml extract below defines the single-host deployment. Both application containers mount the SQLite volume. The API additionally mounts the local runbooks read-only, the ChromaDB volume and the embedding cache. The Docker socket is optional for local-container monitoring, and Ollama provides the self-hosted model path.

```
services:
  api:
    build: {context: ., dockerfile: api/Dockerfile}
    image: ${REGISTRY:-infraguard}/api:latest
    env_file: [.env]
    ports: ["8080:8080"]
    environment:
      - PYTHONPATH=/app
      - DB_PATH=/data/verdicts.db
      - RUNBOOKS_DIR=/app/runbooks
    volumes:
      - ./runbooks:/app/runbooks:ro
      - sqlite_data:/data
      - chroma_data:/app/chroma_db
      - embedding_cache:/root/.cache
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3
  agent:
    build: {context: ., dockerfile: agent/Dockerfile}
    image: ${REGISTRY:-infraguard}/agent:latest
    env_file: [.env]
    environment:
      - PYTHONPATH=/app
      - DB_PATH=/data/verdicts.db
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - sqlite_data:/data
    restart: unless-stopped
    depends_on: [api]
  ollama:
    image: ollama/ollama:latest
    volumes: [ollama_data:/root/.ollama]
    restart: unless-stopped

volumes:
  sqlite_data: {}
  chroma_data: {}
  embedding_cache: {}
  ollama_data: {}
```

APPENDIX D: PROPOSED USABILITY EVALUATION INSTRUMENT

Appendix D contains the task script and System Usability Scale questionnaire prepared for a later practitioner study. The instrument was not administered for Chapter Five. The later study must obtain any ethics approval and participant consent required by University policy.

D.1 Task Script

In a future session, each participant would receive a brief orientation and complete the following tasks while the facilitator records success, time, errors, and requests for help.

1. Log in to the dashboard and state the current overall severity.
2. Open the root-cause and recommended-action sections of a non-OK verdict and paraphrase them.
3. State which integrations are configured by reading the status chips.
4. Ask the runbook assistant a question and confirm a cited answer is returned.
5. Locate a detected threat and trigger the block-IP action (dry-run).
6. Mark a recurring warning as known, and confirm on the next refresh that it is suppressed.
7. Explain what the stale-agent banner indicates.

D.2 System Usability Scale Questionnaire

Participants rate each statement on a five-point scale from 1 (strongly disagree) to 5 (strongly agree). The standard SUS scoring is applied: for odd-numbered items subtract 1 from the response; for even-numbered items subtract the response from 5; sum the ten contributions and multiply by 2.5 to yield a score out of 100 (Brooke, 1996).

1. I think that I would like to use this dashboard frequently.
2. I found the dashboard unnecessarily complex.
3. I thought the dashboard was easy to use.
4. I think that I would need the support of a technical person to be able to use this dashboard.
5. I found the various functions in this dashboard were well integrated.
6. I thought there was too much inconsistency in this dashboard.
7. I would imagine that most people would learn to use this dashboard very quickly.

8. I found the dashboard very cumbersome to use.
9. I felt very confident using the dashboard.
10. I needed to learn a lot of things before I could get going with this dashboard.

A short structured interview may follow the questionnaire and cover effectiveness, efficiency, learnability, and trust. When the study is conducted, the report should include participant characteristics, task results, SUS calculations, qualitative themes, and any interface changes.